Chapter 1

Problem Preliminaries, Complexity, and the Tang-Denardo Paradigm

1.1 Industrial Context and the Uniform SSP (U-SSP)

The paradigm shift toward Flexible Manufacturing Systems (FMS) has revolutionized modern production environments, enabling facilities to process a diverse array of parts with high autonomy. Central to these advanced systems are Computer Numerical Control (CNC) machines, which operate utilizing automated tool changers and finite-capacity tool magazines. While CNC machinery provides the mechanical versatility necessary for high-mix, low-volume manufacturing, it introduces a critical operational bottleneck: the non-productive downtime incurred by continuous cutting tool replacements [**?**, **?**].

Because the physical capacity of a tool magazine is strictly bounded, the machine can only hold a restricted subset of all available tools at any given moment. When a scheduled job requires a tool that is not currently present in the magazine, an automated tool switch must occur. This operation necessitates halting the active production process, extracting a resident tool from the magazine to free a slot, and inserting the newly required tool. In industrial environments where the processing times of individual jobs are relatively short, the cumulative temporal delay caused by these mechanical tool switches can severely degrade the overall machine throughput, neutralizing the theoretical efficiency gains of the FMS [**?**]. Furthermore, excessive tool switching accelerates the physical wear on the automated tool changer's robotic arm, leading to elevated maintenance costs and higher probabilities of mechanical failure.

The computational challenge of determining the optimal operational se-

quence to mitigate these systemic inefficiencies is formalized as the Job Sequencing and Tool Switching Problem (SSP). The fundamental objective of the SSP is to identify a processing permutation for a given set of jobs, concurrently with a corresponding dynamic tool allocation policy, such that the total number of tool switches is strictly minimized [**?**]. Resolving this problem requires coordinating two distinct but highly interdependent decision spaces: the combinatorial sequencing of the jobs and the polyhedral allocation of tools within the finite magazine capacity over discrete time steps.

1.1.1 Mathematical Parameter Definitions

To formally establish the Uniform Job Sequencing and Tool Switching Problem (U-SSP), we must first define the algebraic sets and scalar parameters that dictate the machine's operational environment [**?**].

Let $J = \{1, \ldots, n\}$ define the finite set of jobs that must be sequentially processed, and let $T = \{1, \ldots, m\}$ represent the global set of all available cutting tools in the facility. The CNC machine is equipped with a primary tool magazine of strict integer capacity $b \in \mathbb{Z}^+$, representing the absolute maximum number of tools that can be held simultaneously within the machine's automated carousel. We naturally impose the foundational condition that $m > b$; if this strict inequality did not hold, the entirety of the tool set T could reside in the magazine concurrently, rendering the switching problem trivially solvable with zero continuous transitions.

Each job $j \in J$ requires a specific, predefined subset of tools, denoted as $T_j \subseteq T$, for its successful execution. It is a strict physical prerequisite that the cardinality of each requirement subset does not exceed the magazine capacity, ensuring $|T_j| \leq b$ for all $j \in J$. If this condition were violated, the corresponding job could not be processed without a mid-operation machine halt to exchange tools, a scenario that violates the continuous processing assumptions of standard FMS operational models.

The complete topological map of tool requirements for the entire job set J is conventionally encoded within a sparse binary tool-job incidence matrix $\mathbf{A} \in \{0, 1\}^{m \times n}$. The individual elements of this matrix, a_{tj}, are mathematically defined as follows:

$$a_{tj} = \begin{cases} 1, & \text{if tool } t \text{ is required by job } j \text{ (i.e., } t \in T_j\text{)}, \\ 0, & \text{otherwise.} \end{cases} \tag{1.1}$$

The incidence matrix $\mathbf{A}$ acts as the fundamental data structure governing the SSP. Structurally, the sum of any column j in $\mathbf{A}$ precisely yields the requirement cardinality $|T_j|$ of that specific job, which is strictly bounded from above by b. Conversely, the row sums indicate the global frequency of use for each specific tool across the entire production horizon.

1.1.2 Feasible versus Maximal Configurations and Dummy Tool Padding

At any discrete operational step during the processing of a job sequence, the state of the CNC machine is entirely characterized by the specific subset of tools currently loaded in its magazine. We formally define a *magazine configuration* (or simply a configuration) as a subset $C \subseteq T$.

Definition 1.1. *A configuration C is considered **feasible** for a given job $j \in J$ if and only if two conditions are strictly met: it must contain all tools required to process the job, and its cardinality must respect the physical constraints of the machine. Mathematically, C is feasible for job j if $T_j \subseteq C$ and $|C| \leq b$.*

By this definition, a feasible configuration is permitted to hold fewer tools than the magazine's total capacity (i.e., $|C| < b$). However, in both practical operational scheduling and theoretical graph modeling, it is structurally advantageous to restrict the state space such that the magazine is assumed to be operating at full capacity at all times. We formalize this by defining a *maximal configuration* as a feasible configuration where the magazine is entirely full, satisfying the strict equality $|C| = b$.

To mathematically justify restricting the U-SSP state space exclusively to maximal configurations, we introduce the concept of *dummy tool padding*. We prove that any under-capacity state can be artificially padded with unused tools without ever deteriorating the global objective function.

Lemma 1.2 (Configuration Maximization via Dummy Tool Padding)**.** *For any valid U-SSP instance, an operational sequence containing a non-maximal feasible configuration C (where $|C| < b$) is strictly dominated by, or mathematically equivalent in cost to, a sequence where C is replaced by a padded maximal configuration C' (where $C \subset C'$ and $|C'| = b$).*

Proof. Let $S = (\dots, C_{k-1}, C_k, C_{k+1}, \dots)$ represent a sequence of magazine configurations servicing a given permutation of jobs. Assume the intermediate configuration C_k is feasible for the k-th job in the sequence but is strictly non-maximal, meaning $|C_k| < b$. Let $d(u, v)$ denote the uniform transition cost (the number of required tool insertions) to change the magazine state from configuration u to configuration v. Under the uniform cost assumption, this transition metric is defined by the set difference: $d(u, v) = |C_v \setminus C_u|$.

Because $|C_k| < b$, there exist exactly $b - |C_k|$ unoccupied slots in the magazine during the processing of the k-th job. Let $T_{vacant} \subseteq T \setminus C_k$ be an arbitrary subset of unused tools such that $|T_{vacant}| = b - |C_k|$. We define the padded maximal configuration as $C'_k = C_k \cup T_{vacant}$, ensuring that the strict capacity equality $|C'_k| = b$ holds.

Consider the local transition costs into and out of this updated configuration. The transition cost into the padded configuration from the preceding

state C_{k-1} is $d(C_{k-1}, C'_k) = |C'_k \setminus C_{k-1}|$. Because C'_k is formed merely by filling empty, unutilized slots with tools from T_{vacant}, no previously required tools are forcibly evicted to make room. The setup cost reflects solely the insertion of tools.

More critically, we must examine the transition to the subsequent configuration C_{k+1}. The original required number of tool insertions is $d(C_k, C_{k+1}) = |C_{k+1} \setminus C_k|$. By substituting C_k with the padded configuration C'_k, the new transition cost becomes $d(C'_k, C_{k+1}) = |C_{k+1} \setminus C'_k|$.

From fundamental set theory, since $C_k \subset C'_k$, it strictly follows that the elements in C_{k+1} missing from C'_k must be a subset of the elements in C_{k+1} missing from C_k. That is:

$$(C_{k+1} \setminus C'_k) \subseteq (C_{k+1} \setminus C_k). \tag{1.2}$$

Taking the cardinality of both sides yields the inequality:

$$|C_{k+1} \setminus C'_k| \leq |C_{k+1} \setminus C_k| \implies d(C'_k, C_{k+1}) \leq d(C_k, C_{k+1}). \tag{1.3}$$

Thus, the localized routing cost $d(C_{k-1}, C'_k) + d(C'_k, C_{k+1})$ is strictly less than or equal to $d(C_{k-1}, C_k) + d(C_k, C_{k+1})$. Artificially padding the configuration with dummy tools up to the strict capacity limit b never degrades the sequence's overall switching cost. It either preserves the exact original cost or strictly improves it by coincidentally loading a tool that satisfies a requirement in C_{k+1} or beyond. Consequently, the entire state space of the U-SSP can be restricted to maximal configurations ($|C| = b$) without any loss of global optimality. □

1.2 The Tang-Denardo Decomposition Framework

The fundamental breakthrough in the mathematical analysis of the Uniform SSP was established by Tang and Denardo [**?**], who rigorously demonstrated that the overarching problem naturally and optimally partitions into two nested subproblems: the Sequencing Problem (SP) and the Tooling Problem (TP). This decomposition isolates the distinct combinatorial and polynomial domains of the SSP, providing the foundational architecture for all modern exact and heuristic resolution methods [**?**, **?**].

1.2.1 Formal Definition of the Sequencing Problem (SP)

The Sequencing Problem (SP) governs the discrete temporal ordering in which the set of jobs J is processed by the CNC machine. Mathematically, the SP is defined as the search over the discrete permutation space to minimize the total operational transition costs.

Let S_n denote the symmetric group encompassing all possible permutations of the job set J. A valid job sequence is represented as a bijective

mapping $\sigma \in S_n$, where $\sigma = (\sigma_1, \sigma_2, \ldots, \sigma_n)$, and $\sigma_k \in J$ indicates the specific job processed at the k-th discrete time step. The objective of the SP is to identify a global optimal sequence σ^* that minimizes the total number of required tool switches, operating under the strict assumption that the tool magazine is managed optimally for whichever sequence is chosen.

Definition 1.3 (The Sequencing Problem)**.** *Given the job set J, the tool requirement subsets T_j for all $j \in J$, and the magazine capacity b, find an optimal sequence $\sigma^* \in S_n$ that minimizes the objective function $Z(\sigma)$, where $Z(\sigma)$ denotes the minimal number of tool insertions required to process the sequence σ.*

The SP represents the core combinatorial bottleneck of the SSP and has been formally classified as strongly NP-hard. The complexity of the SP can be intuitively understood through its structural reduction to routing problems. Consider a highly restrictive instance of the SSP where every job requires full magazine capacity, meaning $|T_j| = b$ for all $j \in J$. In this scenario, the transition cost between any two sequentially processed jobs σ_k and σ_{k+1} is strictly defined by the required tool insertions, calculated precisely as $b - |T_{\sigma_k} \cap T_{\sigma_{k+1}}|$.

Under this uniform maximum-capacity condition, the SP collapses directly into a classical Traveling Salesman Problem (TSP) over a complete graph, where the distance metric between any two nodes is exactly the set difference of their tool requirements. Because the TSP is a well-established NP-hard problem in combinatorial optimization, the more generalized Sequencing Problem—where requirement cardinalities can vary and tools can be retained across multiple jobs—is at least as computationally intractable.

1.2.2 Formal Definition of the Tooling Problem (TP)

While the Sequencing Problem addresses the combinatorial arrangement of the jobs, the Tooling Problem (TP) focuses exclusively on the dynamic management of the tool magazine for a *fixed* job sequence σ. Once the permutation $\sigma \in S_n$ is established, the temporal requirements for every tool become strictly deterministic. The objective of the TP is to determine the optimal sequence of magazine configurations that satisfies the tool requirements of σ at every step while strictly minimizing the total number of tool switches.

Let $W_k \subseteq T$ denote the tool magazine configuration at the discrete time step k, precisely when job σ_k is being processed.

Definition 1.4 (The Tooling Problem)**.** *Given a fixed job sequence $\sigma = (\sigma_1, \sigma_2, \ldots, \sigma_n)$ and the corresponding tool requirement subsets T_{σ_k} for all $k \in \{1, \ldots, n\}$, find an optimal sequence of magazine configurations $(W_1, W_2, \ldots, W_n)$*

that minimizes the total number of tool insertions:

$$Z_{TP}(\sigma) = \sum_{k=1}^{n} |W_k \setminus W_{k-1}| \tag{1.4}$$

subject to the structural constraints:

$$T_{\sigma_k} \subseteq W_k \quad \forall k \in \{1, \ldots, n\} \quad \textit{(Job coverage condition)} \tag{1.5}$$

$$|W_k| = b \quad \forall k \in \{1, \ldots, n\} \quad \textit{(Magazine capacity condition)} \tag{1.6}$$

where W_0 defines the initial state of the magazine prior to the processing of the first job.

In stark contrast to the strongly NP-hard Sequencing Problem, the Tooling Problem operates within a highly structured deterministic domain and belongs to the class of problems solvable in polynomial time [**?**]. Because the exact sequence of required tool subsets is known *a priori*, the problem fundamentally reduces to an interval selection task. At any discrete transition from W_{k-1} to W_k where $|T_{\sigma_k} \setminus W_{k-1}| > 0$, the decision-maker must identify exactly which resident tools to evict from the magazine to accommodate the new requirements. As demonstrated by Crama et al. [**?**], the structural properties of this deterministic sequence allow for greedy look-ahead strategies to achieve strict global optimality for the fixed sequence σ.

1.2.3 Structural Interdependence and Algorithmic Complexity

While the Tang-Denardo paradigm elegantly decouples the discrete combinatorial routing from the polynomial-time interval selection, the two subproblems remain fundamentally intertwined. The true complexity of the Uniform SSP arises from this strict structural interdependence: the Sequencing Problem cannot be solved to global optimality using static edge-weight evaluations without continuously querying the Tooling Problem as an oracle.

To rigorously demonstrate why greedy or purely distance-based sequencing permutations fail, we must mathematically analyze the nature of the transition costs. In a classical routing framework, such as the standard Traveling Salesman Problem (TSP), the transition cost c_{ij} between two nodes is static and strictly independent of the sequence history. If one were to apply a greedy nearest-neighbor heuristic to the SSP, the natural static cost estimate between two jobs i and j would be the strict lower bound of required tool insertions, defined by the set union minus the capacity: $c_{ij} = \max\{0, |T_i \cup T_j| - b\}$.

Proposition 1.5 (Failure of Static Greedy Sequencing)**.** *Any sequencing algorithm that constructs a permutation $\sigma \in S_n$ by exclusively optimizing a*

static, localized cost metric c_{ij} between consecutive jobs is fundamentally incapable of guaranteeing global optimality for the U-SSP, as it fails to account for the sequence-dependent state of the tool magazine.

Proof. Consider a partial sequence of jobs $(\sigma_1, \ldots, \sigma_k)$ and the corresponding optimal sequence of magazine configurations $(W_1, \ldots, W_k)$ generated by solving the Tooling Problem. The actual transition cost to process the next job σ_{k+1} is strictly dictated by the cardinality of the set difference: $d(W_k, W_{k+1}) = |W_{k+1} \setminus W_k|$.

Because the magazine capacity b strictly satisfies $b \geq |T_{\sigma_k}|$, the configuration W_k contains a subset of unconstrained slots, specifically $W_k \setminus T_{\sigma_k}$. In an optimal TP resolution, these surplus slots are deliberately populated by tools that are not required by job σ_k, but are retained specifically to satisfy the requirements of future jobs in the sequence, such as $\sigma_{k+1}, \sigma_{k+2}, \ldots, \sigma_n$.

A greedy sequencing algorithm evaluating only the static cost $c_{\sigma_k,\sigma_{k+1}}$ measures merely the disjointness of the core requirement sets T_{σ_k} and $T_{\sigma_{k+1}}$. It operates entirely blind to the dynamically retained tools residing in $W_k \setminus T_{\sigma_k}$. Consequently, the greedy algorithm may select a subsequent job σ_{k+1} that appears locally optimal under the static metric, but actually requires a subset of tools entirely absent from the look-ahead retention set currently residing in W_k.

Selecting this job forces the eviction of highly valuable retained tools to make room for the new requirements. This localized decision inflicts severe, cascading setup penalties in subsequent transitions to σ_{k+2} and beyond, destroying global optimality.

Therefore, the transition cost between any two jobs in the SSP is dynamically sequence-dependent. The true cost of appending job σ_{k+1} is a strict mathematical function of the entire preceding permutation $(\sigma_1, \ldots, \sigma_k)$, which dictates the precise optimal composition of W_k. Thus, any exact resolution of the SP mandates that the TP must be optimally solved—or its exact logic implicitly bounded—to accurately evaluate the permutation, confirming their unbreakable structural interdependence. □

1.3 Interval Matrices and the Keep Tool Needed Soonest (KTNS) Policy

1.3.1 Definition of 0-Blocks and Interval Properties

With the Tooling Problem (TP) securely classified within the polynomial-time domain for any fixed job sequence $\sigma = (\sigma_1, \ldots, \sigma_n)$, we now mathematically characterize its exact resolution strategy. The operational mechanics of the TP are best analyzed by observing the discrete temporal requirement patterns of individual tools across the fixed sequence.

Let $\mathbf{A}^\sigma \in \{0,1\}^{m\times n}$ denote the permuted tool-job incidence matrix, where the columns of the original matrix $\mathbf{A}$ have been rearranged to exactly match the processing sequence σ. An element $a^\sigma_{t,k} = 1$ if tool $t \in T$ is required for the k-th job in the sequence (i.e., $t \in T_{\sigma_k}$), and 0 otherwise.

Definition 1.6 (0-Block)**.** *For a specific tool $t \in T$ and a fixed sequence σ, a* ***0-block*** *is defined as a maximal contiguous sequence of zeros in the t-th row of the permuted incidence matrix $\mathbf{A}^\sigma$, bounded by ones. Mathematically, the discrete interval $[k_1, k_2]$ forms a 0-block if $a^\sigma_{t,k} = 0$ for all $k \in \{k_1, \dots, k_2\}$, while $a^\sigma_{t,k_1-1} = 1$ and $a^\sigma_{t,k_2+1} = 1$.*

Operationally, a 0-block represents an idle temporal interval during which tool t is strictly not required by the active jobs, yet is deterministically known to be required by a future job σ_{k_2+1}. During this interval, the decision-maker faces a critical choice: either evict tool t from the magazine at step $k_1 - 1$ to free a capacity slot for other required tools (which necessitates a mechanical reload penalty at step $k_2 + 1$), or retain tool t continuously in the magazine throughout the interval $[k_1, k_2]$.

The act of retaining an unrequired tool across an idle interval to avoid a future setup penalty is mathematically referred to as *flipping a 0-block*. When a 0-block is flipped, the effective presence of the tool in the magazine is modeled by temporarily converting the sequence of zeros into ones in the decision state matrix. If every 0-block for a given tool is flipped, the row corresponding to that tool exhibits the Consecutive Ones Property (COP), meaning all 1s appear completely contiguously. Matrices where all rows satisfy the COP are formally known as interval matrices.

Crama et al. [**?**] demonstrated that the tooling subproblem can be formulated as a search for the optimal combination of 0-blocks to flip such that the strict magazine capacity b is never violated at any column index k. The uniform transition metric forces the optimization algorithm to greedily maximize the number of flipped 0-blocks to avoid redundant tool switches. This interval matrix structure, and its inherent total unimodularity, mathematically underpins the logic of the renowned Keep Tool Needed Soonest (KTNS) policy [**?**].

1.3.2 The Formal KTNS Algorithmic Logic

Given the fixed sequence of jobs $\sigma = (\sigma_1, \sigma_2, \dots, \sigma_n)$, the Keep Tool Needed Soonest (KTNS) policy provides a computationally efficient and exact resolution to the Tooling Problem. The fundamental axiom of KTNS relies on a greedy look-ahead mechanism: when the magazine is at full capacity b, and a resident tool must be evicted to accommodate a newly required tool for the current job, the optimal choice is strictly to evict the tool whose next required usage is furthest in the future [**?**].

To formalize this look-ahead logic, we define an interval tracking function $L(i,k)$ which computes the step index of the next imminent requirement for tool $i \in T$ at or after the discrete time step k:

$$L(i,k) = \min\{t \geq k \mid i \in T_{\sigma_t}\}. \tag{1.7}$$

If tool i is no longer required for the remainder of the sequence σ, we set $L(i,k) = n+1$, representing a bounding time step safely beyond the planning horizon.

Operating on this tracking function, the KTNS policy iterates sequentially through the job permutation σ and dynamically updates the magazine configurations W_k. The procedure is formally detailed in Algorithm 1.

Algorithm 1 The Keep Tool Needed Soonest (KTNS) Policy

Require: A job permutation $\sigma \in S_n$, requirement subsets $T_{\sigma_k} \subseteq T$, magazine capacity b.

Ensure: The optimal sequence of maximal configurations $(W_1, \ldots, W_n)$ and the minimum total tool switches $Z_{TP}(\sigma)$.

```
Initialization: Compute L(i,k) for all i ∈ T and k ∈ {1,...,n}.
Construct W_1 by first inserting all required tools T_{σ_1}.
Pad W_1 to strict capacity b using exactly b − |T_{σ_1}| tools i ∈ T \ T_{σ_1} that
    possess the smallest values of L(i,1).
Z_TP ← 0
for k = 2 to n do
    W_k ← W_{k−1}                    ▷ Inherit the previous magazine configuration
    for each required tool j ∈ T_{σ_k} do
        if j ∉ W_k then
            Select tool p ∈ W_k \ T_{σ_k} such that p = arg max_i L(i,k).    ▷
    KTNS Eviction Logic
            W_k ← (W_k \ {p}) ∪ {j}
        end if
    end for
    Z_TP ← Z_TP + |W_k \ W_{k−1}|          ▷ Accumulate transition cost
end for
return (W_1,...,W_n) and Z_TP
```

By executing the replacement logic delineated in Algorithm 1, the KTNS policy explicitly avoids early insertions and prevents the eviction of tools that will be required imminently. This behavior maps directly to the maximization of flipped 0-blocks within the permuted incidence matrix $\mathbf{A}^\sigma$, as tools spanning short idle intervals are naturally retained. This ensures the physical magazine operates at maximum mathematical efficiency without incurring any avoidable setup penalties.

1.3.3 Theorem and Proof of KTNS Optimality

The theoretical validity of the KTNS policy hinges on establishing that the greedy, localized decision of maximizing the forward interval before a tool's next usage never degrades the global objective function. We formalize this optimality through a rigorous exchange argument.

Theorem 1.7 (Global Optimality of KTNS)**.** *For any fixed job sequence $\sigma \in S_n$ and any uniform magazine capacity b, the Keep Tool Needed Soonest (KTNS) policy yields a sequence of magazine configurations $(W_1, \ldots, W_n)$ that strictly minimizes the total number of tool switches $Z_{TP}(\sigma)$.*

Proof. We proceed via an exchange argument. Let g^* be an optimal tooling policy that minimizes the total number of tool insertions for a fixed sequence σ. Assume that g^* diverges from the KTNS policy at some discrete time step k.

At step k, a tool must be evicted from the magazine W_k to accommodate a newly required tool for the subsequent job. According to the KTNS logic, the optimal tool to evict is $t \in W_k \setminus T_{\sigma_{k+1}}$ such that its next requirement is furthest in the future, meaning $L(t, k) = \max_i L(i, k)$. Let $q = L(t, k)$.

Suppose, however, that the optimal policy g^* diverges by instead evicting a different tool $s \in W_k \setminus T_{\sigma_{k+1}}$, while retaining tool t. Because t is the KTNS choice, it strictly follows that s is required sooner than t, giving the strict inequality $p = L(s, k) < L(t, k) = q$.

Because g^* evicted s at step k, it will be forced to incur a mechanical setup penalty to re-insert tool s exactly at step p, the point of its next requirement. During the interval (k, p), policy g^* retains tool t in the magazine, occupying a capacity slot even though t is not required until step q.

Let us construct an alternative feasible policy g' which is identical to g^* for all steps prior to k. At step k, policy g' aligns with KTNS: it evicts t and retains s. During the interval (k, p), policy g' utilizes the capacity slot freed by t to hold s. Consequently, at step p, policy g' does not need to insert s, saving exactly one setup operation compared to g^*.

At step p, policy g^* must evict some tool, say u, to make room for s. Policy g' can mimic this by evicting u to make room for the tool it must now load, or it can simply hold the slot if no loading is required. Ultimately, at step q, policy g' must incur a setup penalty to re-insert tool t, an operation that policy g^* might avoid if it successfully retained t throughout the entire interval (k, q).

However, because $p < q$, policy g' deferred its setup penalty further into the future than g^*. The configuration sequence under g' maintains strictly equivalent or greater flexibility during the interval (p, q) because it is not burdened by the presence of t until absolutely necessary. Therefore, the

total number of tool switches under the modified policy g' satisfies $Z_{TP}^{g'}(\sigma) \leq Z_{TP}^{g^*}(\sigma)$.

If $Z_{TP}^{g'}(\sigma) < Z_{TP}^{g^*}(\sigma)$, it contradicts the assumption that g^* is uniquely optimal. If $Z_{TP}^{g'}(\sigma) = Z_{TP}^{g^*}(\sigma)$, it proves that aligning the eviction choice with the KTNS rule at step k preserves global optimality. By iteratively applying this exchange argument across all steps $k \in \{1, \ldots, n\}$, we can transform any optimal policy into the KTNS policy without ever increasing the total cost. Thus, the KTNS policy is globally optimal for the Tooling Problem. □

1.3.4 Visualizing Dynamic Tool Retention: 0-Block Flipping

To concretely illustrate the mathematical dynamics of the KTNS policy, we construct a deterministic processing scenario. Consider a uniform U-SSP instance defined by a job set $J = \{1, 2, 3, 4, 5\}$, a tool set $T = \{t_1, \ldots, t_7\}$, and a strict magazine capacity of $b = 4$. The tool requirements T_j for each job are defined *a priori* by the permuted incidence matrix.

Figure 1.1 provides a comprehensive timeline graphic detailing the exact state of the tool magazine across this sequence. The visual explicitly demonstrates the operational mechanism of "0-block flipping," where tools are purposefully retained across idle intervals to prevent future mechanical setup penalties.

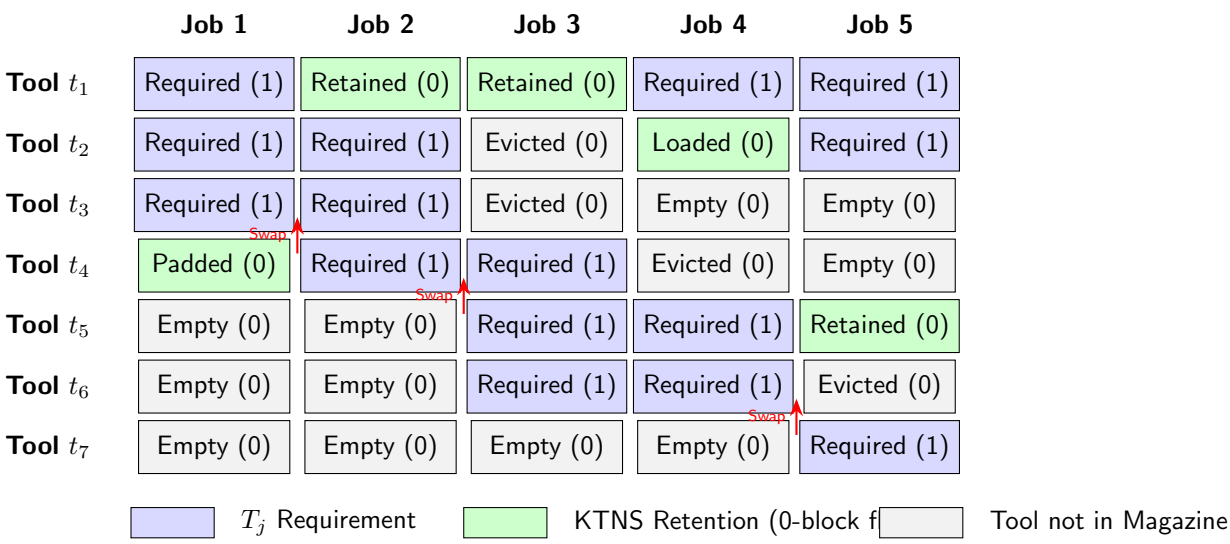

Figure 1.1: Evolution of the magazine configurations under the KTNS policy for a 5-job sequence with capacity $b = 4$. The graphic explicitly highlights the retention of tool t_1 across jobs 2 and 3, avoiding subsequent setups.

The mathematical elegance of the KTNS policy is immediately observ-

able in the handling of tool t_1. Although t_1 is strictly required only at jobs 1, 4, and 5, it occupies a zero-interval during the processing of jobs 2 and 3. Because the magazine capacity $b = 4$ is sufficient to accommodate the requirements of jobs 2 and 3 (which only strictly require three tools each) alongside the retention of t_1, the KTNS algorithm explicitly "flips" this 0-block. By artificially treating t_1 as a requirement during the interval $[1, 2]$, the algorithm mathematically forces the system to preserve the physical setup, successfully avoiding a redundant eviction and re-insertion.

However, the KTNS logic is rigidly bounded by the physical capacity limits. Consider tool t_2. It is required at jobs 1, 2, and 5. During the processing of job 3, tool t_2 enters a 0-block. Ideally, the algorithm would retain t_2 across jobs 3 and 4 to satisfy the requirement at job 5. Yet, job 3 explicitly requires $T_3 = \{t_4, t_5, t_6\}$. Loading these three tools, combined with the retention of the higher-priority tool t_1 (which is needed imminently at job 4), strictly saturates the magazine capacity ($|C_3| = 4$). The algorithm is therefore mathematically compelled to evict tool t_2 at step 3, incurring a mandatory future setup penalty to reload it prior to job 5.

This localized interval analysis confirms the global optimality established in Theorem 1.1: for any fixed permutation, the KTNS interval-matrix formulation guarantees that tool setups are induced exclusively by the unavoidable mathematical conflicts between capacity bounds and intersecting requirement sets, entirely eliminating arbitrary or redundant mechanical operations.

Chapter 2

Sequence-Based Formulations and the Symmetry Menace

2.1 Early Compact Integer Linear Programs

While the Tang-Denardo decomposition and the KTNS policy effectively resolve the Tooling Problem for a fixed sequence, identifying the optimal global sequence inherently requires evaluating the combinatorial space of all possible permutations. Early attempts to solve the Job Sequencing and Tool Switching Problem (SSP) exactly to global optimality relied on formulating the entire problem as a single, compact Integer Linear Program (ILP). These formulations attempted to simultaneously optimize the discrete job permutation and the dynamic tool allocations by explicitly tracking the state of the machine at every position in the sequence.

2.1.1 The Position-Based Formulation of Tang and Denardo

The inaugural exact mathematical formulation for the U-SSP was proposed by Tang and Denardo [**?**]. This model relies on a positional assignment logic, wherein the discrete temporal processing steps are explicitly modeled as an ordered set of positions.

Let $K = \{1, 2, \ldots, n\}$ denote the set of discrete positions in the processing sequence. The formulation defines three distinct classes of binary decision variables to capture the job permutation, the tool magazine state, and the mechanical tool switches, respectively:

- $u_{jk} \in \{0, 1\}$: A binary assignment variable equal to 1 if job $j \in J$ is assigned to position $k \in K$ in the processing sequence, and 0 otherwise.
- $v_{kt} \in \{0, 1\}$: A binary state variable equal to 1 if tool $t \in T$ is present

in the tool magazine during the processing of the job at position $k \in K$, and 0 otherwise.

- $w_{kt} \in \{0, 1\}$: A binary transition variable equal to 1 if tool $t \in T$ is inserted into the magazine at position $k \in K$ (i.e., immediately prior to processing the k-th job), and 0 otherwise.

Given the tool-job incidence matrix $\mathbf{A}$ where $a_{tj} \in \{0, 1\}$ indicates whether tool t is required by job j, the complete position-based ILP is formulated as follows:

$$\text{Minimize} \quad Z = \sum_{k=1}^{n} \sum_{t=1}^{m} w_{kt} \tag{2.1}$$

$$\text{subject to} \quad \sum_{k=1}^{n} u_{jk} = 1 \qquad \forall j \in J \tag{2.2}$$

$$\sum_{j=1}^{n} u_{jk} = 1 \qquad \forall k \in K \tag{2.3}$$

$$v_{kt} \geq \sum_{j=1}^{n} a_{tj} u_{jk} \qquad \forall k \in K, \forall t \in T \tag{2.4}$$

$$\sum_{t=1}^{m} v_{kt} \leq b \qquad \forall k \in K \tag{2.5}$$

$$w_{kt} \geq v_{kt} - v_{k-1,t} \qquad \forall k \in K, \forall t \in T \tag{2.6}$$

$$u_{jk}, v_{kt}, w_{kt} \in \{0, 1\} \qquad \forall j \in J, \forall k \in K, \forall t \in T \tag{2.7}$$

The objective function (2.1) minimizes the total number of tool insertions across the entire processing horizon. Constraints (2.2) and (2.3) represent the classical assignment problem constraints, strictly ensuring that the assignment mapping from the job set J to the position set K constitutes a valid bijective permutation $\sigma \in S_n$.

Constraint (2.4) strictly enforces the operational feasibility of the configuration at each step: if job j is assigned to position k ($u_{jk} = 1$) and it requires tool t ($a_{tj} = 1$), then tool t is mathematically forced to be present in the magazine at position k ($v_{kt} = 1$). Constraint (2.5) models the physical hardware limitation, bounding the summation of resident tools at any position by the magazine capacity b.

Finally, constraint (2.6) dictates the tool transition logic. It forces the insertion variable w_{kt} to take the value of 1 if tool t is present at position k but was strictly absent at the preceding position $k - 1$. To ensure completeness, the boundary condition at the start of the sequence assumes the magazine is initially empty, establishing $v_{0,t} = 0$ for all $t \in T$. Because the

objective function exerts a strict downward pressure on the continuous variables w_{kt}, they will naturally bound exactly to $\max\{0, v_{kt} - v_{k-1,t}\}$ without requiring explicit equality constraints or integer declarations for w_{kt}.

2.1.2 The Arc-Flow Formulation of Laporte et al.

To circumvent the severe computational inefficiencies associated with positional indexing, Laporte, Salazar-González, and Semet [**?**] proposed a fundamentally different integer linear program based on an arc-flow topological framework. Rather than mapping jobs to absolute discrete time steps, this model relies on relative sequencing, thereby embedding the Sequencing Problem within a classical Asymmetric Traveling Salesman Problem (ATSP) structure.

Let $J_0 = J \cup \{0\}$ denote the augmented job set, where the dummy job 0 represents the initial and terminal state of the machine magazine (which is strictly assumed to be empty). The problem is defined on a complete directed graph where the vertices correspond to the jobs in J_0. The formulation introduces three interconnected sets of binary decision variables:

- $x_{ij} \in \{0, 1\}$: A binary routing variable equal to 1 if and only if job j is processed strictly immediately after job i, for all $i, j \in J_0$ with $i \neq j$.

- $y_{tj} \in \{0, 1\}$: A binary magazine state variable equal to 1 if and only if tool $t \in T$ is present in the magazine during the processing of job $j \in J_0$.

- $z_{tj} \in \{0, 1\}$: A binary transition variable equal to 1 if and only if tool $t \in T$ is mechanically inserted into the magazine immediately prior to the processing of job $j \in J_0$.

Given the tool-job incidence matrix parameter a_{tj}, the foundational arc-

flow ILP is formulated to minimize the sum of all tool insertions:

$$\text{Minimize} \quad Z_{arc} = \sum_{j \in J_0} \sum_{t=1}^{m} z_{tj} \tag{2.8}$$

$$\text{subject to} \quad \sum_{i \in J_0 \setminus \{j\}} x_{ij} = 1 \qquad \forall j \in J_0 \tag{2.9}$$

$$\sum_{j \in J_0 \setminus \{i\}} x_{ij} = 1 \qquad \forall i \in J_0 \tag{2.10}$$

$$\sum_{i \in S} \sum_{j \in S \setminus \{i\}} x_{ij} \leq |S| - 1 \quad \forall S \subseteq J, |S| \geq 2 \tag{2.11}$$

$$y_{tj} \geq a_{tj} \qquad \forall j \in J, \forall t \in T \tag{2.12}$$

$$\sum_{t=1}^{m} y_{tj} \leq b \qquad \forall j \in J_0 \tag{2.13}$$

$$z_{tj} \geq y_{tj} - y_{ti} + x_{ij} - 1 \quad \forall i, j \in J_0, i \neq j, \forall t \in T \tag{2.14}$$

$$x_{ij}, y_{tj}, z_{tj} \in \{0, 1\} \qquad \forall i, j \in J_0, \forall t \in T \tag{2.15}$$

Constraints (2.9), (2.10), and (2.11) constitute the classical routing constraints. The in-degree and out-degree equations guarantee that every job is preceded and succeeded by exactly one other job, while the Subtour Elimination Constraints (SECs) ensure that the variables x_{ij} form a single, continuous Hamiltonian cycle covering all vertices in J_0.

Constraint (2.12) guarantees that the tool requirements are natively satisfied at each node, enforcing that if a tool is required ($a_{tj} = 1$), it must be physically present ($y_{tj} = 1$). Constraint (2.13) acts as the structural capacity bound for the magazine.

The critical mathematical linkage between the sequence routing and the magazine state is governed by constraint (2.14). By utilizing the bound $+x_{ij} - 1$, the formulation gracefully avoids weak "Big-M" relaxations. If job j does not immediately follow job i (i.e., $x_{ij} = 0$), the right-hand side becomes $y_{tj} - y_{ti} - 1 \leq 0$, rendering the constraint inactive for the non-negative variable z_{tj}. However, if job j strictly follows job i ($x_{ij} = 1$), the constraint enforces $z_{tj} \geq y_{tj} - y_{ti}$. Consequently, if tool t is present at job j but was absent during job i, the algorithm mathematically triggers a mandatory setup penalty by forcing $z_{tj} = 1$.

2.1.3 Objective Lifting and Polyhedral Strengthening Inequalities

To mitigate the severe computational intractability caused by the weak linear relaxation bounds of the standard arc-flow model, the formulation can be tightened through polyhedral strengthening and objective lifting [**?**].

The fundamental mathematical weakness of the transition constraint (2.14) lies in its isolated, tool-by-tool evaluation. When the domain is relaxed to continuous variables, the solver can arbitrarily distribute fractional values of tool presences y_{tj} to artificially satisfy the constraint without triggering the discrete insertion penalty z_{tj}. To structurally bind the aggregate transition costs directly to the combinatorial routing decisions, we introduce a family of *Transition Lower Bound Inequalities*.

For any two jobs $i, j \in J$, if job j is processed strictly immediately after job i (i.e., $x_{ij} = 1$), the physical mechanics of the CNC machine dictate that the magazine must undergo an absolute minimum number of tool insertions simply to satisfy the joint requirements. This localized lower bound is mathematically defined by the capacity deficit of the set union: $\max\{0, |T_i \cup T_j| - b\}$. By aggregating this deterministic penalty across all potential predecessors $i \in J_0$, we formulate a lifted bounding inequality for the total insertions required at node j:

$$\sum_{t \in T} z_{tj} \geq \sum_{i \in J_0 \setminus \{j\}} \max\{0, |T_i \cup T_j| - b\} x_{ij} \quad \forall j \in J. \tag{2.16}$$

This inequality acts as a lifted objective bound. It provides a strict, sequence-dependent polyhedral floor, directly translating the hypergraph overlap of the tool requirement sets into a hard mathematical cost within the routing domain, preventing the LP relaxation from entirely bypassing setup penalties.

Furthermore, the generic capacity inequality (2.13) can be structurally lifted to an exact equality. By invoking the Configuration Maximization Lemma (Lemma 1.2), we have theoretically established that maintaining the tool magazine strictly at its capacity limit b never degrades global optimality. Thus, the polyhedral space is substantially tightened by replacing the inequality with the strict conservation equation:

$$\sum_{t=1}^{m} y_{tj} = b \quad \forall j \in J_0. \tag{2.17}$$

Enforcing this equality natively eliminates the massive sub-space of symmetries associated with under-utilized, partially empty magazine states, forcing the linear relaxation to distribute exactly b fractional units of tool presence across the available tools at every single node.

To further restrict the fractional space, the arc-flow formulation can be augmented with generalized 2-matching inequalities and comb inequalities natively adapted from the polyhedral studies of the symmetric Traveling Salesman Problem. However, while the inclusion of these advanced facet-defining inequalities and lifted transition bounds significantly tightens the initial LP relaxation, the underlying framework inherently retains the sequence-based architecture of the U-SSP. As we will mathematically prove

in Section 2.3, attempting to track individual tool lifespans across relative, sequence-dependent node variables ultimately induces profound mathematical degeneracy that no polynomial class of valid inequalities can fully resolve.

2.2 Advanced Compact Formulations

The inherent weakness of the position-based and basic arc-flow models lies in their vulnerability to massive fractional degeneracy during the linear relaxation phase. Because these early models evaluate tool transitions entirely locally (i.e., step-by-step or node-to-node), the solver can artificially fractionalize the presence of a tool to satisfy capacity constraints without triggering the discrete insertion penalties. To counter this, advanced compact models shifted the mathematical perspective from tracking local transitions to tracking the continuous "lifespan" of a tool across the sequence.

2.2.1 Linear Ordering and Catanzaro's Overlap Constraints

The pinnacle of sequence-based compact modeling is represented by the formulations developed by Catanzaro et al. [**?**], particularly their most computationally effective model, known in the literature as Formulation 5. To eliminate the symmetries induced by absolute positional indexing, this formulation models the Sequencing Problem using a strict linear ordering framework.

Let $x_{ij} \in \{0, 1\}$ be a binary linear ordering variable that equals 1 if job $i \in J$ is processed strictly before job $j \in J$ (not necessarily immediately), and 0 otherwise. The fundamental combinatorial structure of the sequence is enforced via classical tournament and transitivity constraints:

$$x_{ij} + x_{ji} = 1 \quad \forall i, j \in J, i \neq j \tag{2.18}$$

$$x_{ij} + x_{jk} + x_{ki} \leq 2 \quad \forall i, j, k \in J, i \neq j \neq k \tag{2.19}$$

Rather than utilizing a local step-variable w_{kt} to penalize isolated insertions, Catanzaro et al. model the operational logic of the KTNS policy directly within the continuous routing space. If a tool t is required by job i and subsequently required by job j, the decision-maker can choose to retain tool t in the magazine during all intermediate jobs processed between i and j.

To capture this, we define a binary retention variable $y^t_{ij} \in \{0, 1\}$ for every tool $t \in T$ and for every pair of jobs $i, j \in J$ such that $t \in T_i \cap T_j$. The variable $y^t_{ij} = 1$ if and only if job i is processed before job j ($x_{ij} = 1$), tool t is continuously retained in the magazine between i and j, and no other job requiring tool t is processed between i and j.

The objective function seeks to maximize the number of avoided tool switches, which is mathematically equivalent to maximizing the number of

times a tool is successfully retained between its required operations:

$$\text{Maximize} \quad Z_{retention} = \sum_{t \in T} \sum_{i \in J: t \in T_i} \sum_{j \in J: t \in T_j, i \neq j} y_{ij}^t \tag{2.20}$$

The critical innovation of this formulation lies in the *Overlap Capacity Constraints*. For any given job $k \in J$, the tool magazine must physically accommodate the strict requirements of job k, occupying exactly $|T_k|$ slots. The remaining $b - |T_k|$ slots are available to hold tools that are not required by k, but are being retained to bridge the gap between some prior job i and some future job j.

A tool $t \notin T_k$ overlaps job k if and only if i precedes k ($x_{ik} = 1$), k precedes j ($x_{kj} = 1$), and the retention variable y_{ij}^t is active. This overlap logic is rigorously bounded by the following polyhedral inequality:

$$\sum_{t \notin T_k} \sum_{i \in J: t \in T_i} \sum_{j \in J: t \in T_j} y_{ij}^t (x_{ik} + x_{kj} - 1) \leq b - |T_k| \quad \forall k \in J. \tag{2.21}$$

Constraint (2.21) ensures that the capacity of the magazine is never violated by the simultaneous retention of unrequired tools across any specific job k. The non-linear product $y_{ij}^t(x_{ik} + x_{kj} - 1)$ strictly equals 1 only when job k is structurally sandwiched between i and j in the linear order, and tool t is actively bridging them. In the exact MILP implementation, this product is effortlessly linearized using standard bounding inequalities. By directly tracking tool life spans, Formulation 5 significantly tightens the linear relaxation compared to position-based models, establishing a higher lower-bound floor for exact branch-and-bound solvers [**?**].

2.2.2 The Multi-Commodity Flow Formulation (SSPMF)

An alternative topological perspective to the linear ordering model is to conceptualize the continuous retention of tools across a job sequence as a multi-commodity network flow problem. By treating each individual tool $t \in T$ as a distinct commodity that must be routed through the machine's operational states, the SSP Multi-Commodity Flow (SSPMF) formulation explicitly maps the physical lifespan of tools onto the combinatorial routing decisions.

To formalize the SSPMF, we define a complete directed graph over the augmented job set $J_0 = J \cup \{0\}$, where the dummy node 0 represents the empty initial and terminal states of the tool magazine. The formulation retains the standard binary routing variables $x_{ij} \in \{0, 1\}$ to dictate the job sequence, and the binary state variables $y_{tj} \in \{0, 1\}$ to indicate the presence of tool t during the processing of job j. Furthermore, it introduces the setup penalty variables $z_{tj} \geq 0$, representing the mechanical insertion of tool t at node j.

The core innovation of the SSPMF rests on the introduction of the commodity flow variables:

- $f_{ij}^t \in \{0,1\}$: A binary flow variable equal to 1 if and only if tool $t \in T$ is continuously retained in the magazine during the direct transition from job i to job j, and 0 otherwise.

The objective remains the minimization of the total required insertions, $\sum_{j \in J} \sum_{t \in T} z_{tj}$. Alongside the classical routing degree constraints and Generalized Subtour Elimination Constraints (GSECs) over the variables x_{ij}, the SSPMF establishes the precise operational limits of the tool magazine through the following capacity and flow conservation equations:

$$y_{tj} \geq a_{tj} \qquad \forall j \in J, \forall t \in T \tag{2.22}$$

$$\sum_{t=1}^{m} y_{tj} \leq b \qquad \forall j \in J_0 \tag{2.23}$$

$$f_{ij}^t \leq x_{ij} \qquad \forall i,j \in J_0, i \neq j, \forall t \in T \tag{2.24}$$

$$\sum_{t=1}^{m} f_{ij}^t \leq b \cdot x_{ij} \qquad \forall i,j \in J_0, i \neq j \tag{2.25}$$

$$z_{tj} \geq y_{tj} - \sum_{i \in J_0 \setminus \{j\}} f_{ij}^t \qquad \forall j \in J, \forall t \in T \tag{2.26}$$

$$x_{ij}, y_{tj}, f_{ij}^t \in \{0,1\} \qquad \forall i,j \in J_0, \forall t \in T \tag{2.27}$$

Constraints (2.22) and (2.23) mandate the standard nodal feasibility logic: required tools must be present, and the total presence at any node cannot exceed the magazine capacity b.

Constraint (2.24) structurally binds the tool commodities to the sequence graph, dictating that a tool can only "flow" between two jobs if the direct routing edge is active ($x_{ij} = 1$). Constraint (2.25) ensures that the total volume of all commodities flowing across any active routing edge respects the strict physical capacity of the magazine carrier.

The critical nexus of the formulation is the flow conservation inequality (2.26). It mathematically balances the state of the magazine by defining how a tool arrives at node j. If tool t is present at job j ($y_{tj} = 1$), it must have either arrived via a continuous retention flow from the preceding job (in which case the sum of incoming flows $\sum_i f_{ij}^t = 1$), or it must be mechanically loaded at node j. If the incoming flow is zero, the right-hand side evaluates to 1, strictly forcing the setup penalty variable $z_{tj} \geq 1$.

By forcing the LP solver to explicitly distribute fractional flow paths for every tool along the selected routing arcs, the SSPMF significantly mitigates the isolated node-to-node degeneracies of the basic arc-flow models.

The explicit tracking of tool lifespans as continuous network commodities establishes a tighter polyhedral floor, allowing modern exact solvers to prune significantly larger portions of the branch-and-bound tree. However, despite these advancements, all sequence-based compact models ultimately remain susceptible to the underlying combinatorial symmetries of the problem, a fundamental limitation that necessitates a topological shift toward non-compact configuration domains.

2.3 Mathematical Proof of Symmetry and LP Collapse

2.3.1 Analysis of Fractional Assignment in the Linear Relaxation

The practical efficacy of any exact integer linear programming framework, particularly those solved via classical branch-and-bound or branch-and-cut methodologies, is strictly dictated by the tightness of its continuous linear relaxation (LP). A tight LP lower bound accelerates the pruning of suboptimal nodes in the search tree, whereas a weak or trivial lower bound renders the exponential search space computationally intractable [**?**, **?**]. For the sequence-based compact formulations of the Uniform SSP—including the position-based, arc-flow, and multi-commodity models—the transition from the discrete binary domain to the continuous fractional domain exposes a profound structural vulnerability: the artificial satisfaction of capacity and coverage constraints via fractional assignment.

To rigorously analyze this mathematical collapse, consider the linear relaxation of the foundational Tang and Denardo position-based model presented in Section 2.1. The discrete job-to-position assignment variables $u_{jk} \in \{0, 1\}$ are relaxed to the continuous domain $u_{jk} \geq 0$. Under this relaxation, the objective function seeks to minimize the continuous tool insertion variables $w_{kt} \geq 0$, subject to the relaxed transition bounds $w_{kt} \geq v_{kt} - v_{k-1,t}$.

Because the objective function exerts a strict minimization pressure on w_{kt}, the LP solver's optimal strategy is to configure the fractional tool presence variables v_{kt} such that the state gradient $v_{kt} - v_{k-1,t}$ is universally non-positive, or ideally, strictly zero across all discrete positions $k \in K$. In the discrete physical reality of the machine, maintaining a static magazine state $v_{kt} = v_{k-1,t}$ across consecutive jobs is impossible unless those jobs require the exact same subset of tools, bounded strictly by the physical capacity constraint $\sum_{t \in T} v_{kt} \leq b$.

However, in the continuous polyhedral space, the solver elegantly exploits the assignment constraints to bypass this physical bottleneck. By uniformly distributing a fractional assignment of every job across every position in the sequence, such that $u_{jk} = \frac{1}{n}$ for all $j \in J$ and $k \in K$, the classical assignment

equations are perfectly satisfied without violation:

$$\sum_{k=1}^{n} \frac{1}{n} = 1 \quad \forall j \in J, \qquad \sum_{j=1}^{n} \frac{1}{n} = 1 \quad \forall k \in K. \tag{2.28}$$

This uniform fractional dispersion acts as a mathematical smoothing function over the highly heterogeneous temporal tool requirements. The mandatory coverage constraint dictates the required tool presence at any position k:

$$v_{kt} \geq \sum_{j=1}^{n} a_{tj} u_{jk} = \sum_{j=1}^{n} a_{tj} \left(\frac{1}{n}\right) = \frac{1}{n} \sum_{j=1}^{n} a_{tj} \quad \forall k \in K, \forall t \in T. \tag{2.29}$$

Here, the sum $\sum_{j=1}^{n} a_{tj}$ represents the global frequency of requirement for tool t across the entire production horizon. To minimize the continuous holding cost and satisfy the constraints, the solver sets the fractional magazine state for tool t at every single position k to exactly its proportional frequency: $v_{kt} = \frac{1}{n} \sum_{j=1}^{n} a_{tj}$.

Because this fractional tool distribution is entirely static and perfectly identical across all sequential positions $k \in K$, the difference between any two consecutive magazine states evaluates exactly to zero:

$$v_{kt} - v_{k-1,t} = 0 \implies w_{kt} = 0 \quad \forall k \in K, \forall t \in T. \tag{2.30}$$

This analytical phenomenon demonstrates that the continuous LP relaxation completely uncouples the temporal tool requirements from the physical capacity limits. By fractionalizing the sequence, the solver is able to perfectly "smear" the jobs across the time horizon, blending the isolated peaks of tool requirements into a flat, continuous average that never triggers a discrete transition penalty.

2.3.2 Formal Derivation of 0-Block Permutational Symmetry

Beyond the continuous fractional degeneracy of the linear relaxation, the discrete integer space of sequence-based compact models is profoundly afflicted by structural symmetry. Even when variables are constrained to the binary domain $u_{jk}, v_{kt} \in \{0, 1\}$, the formulation must implicitly search through an exponentially large plateau of redundant magazine configurations. This phenomenon is mathematically rooted in the independent evaluation of 0-block retention decisions.

To formalize this symmetric explosion, we analyze the behavior of the magazine state variables across idle intervals. Consider a fixed job permutation $\sigma \in S_n$ and its associated permuted incidence matrix $\mathbf{A}^{\sigma}$.

Definition 2.1 (Independent 0-Block Set)**.** *Let $\mathcal{B} = \{B_1, B_2, \ldots, B_k\}$ be a set of k distinct 0-blocks occurring across various tools during a contiguous sub-sequence of jobs $[\sigma_p, \ldots, \sigma_q]$. The set $\mathcal{B}$ is considered independent if the concurrent retention of all corresponding tools across their respective intervals does not strictly violate the magazine capacity bound b at any position.*

In the discrete physical reality of the machine, the Keep Tool Needed Soonest (KTNS) policy resolves independent 0-blocks deterministically: if capacity permits, all k 0-blocks should be greedily flipped (i.e., the tools retained) to prevent future insertion penalties. However, sequence-based compact models, such as the position-based and arc-flow formulations, do not inherently possess this look-ahead oracle. Instead, they model the presence of a tool t at position p as an isolated binary decision variable v_{pt} (or y_{tj} in arc-flow).

Theorem 2.2 (2^k Permutational Symmetry of the State Space)**.** *For any fixed sequence containing a set of k independent 0-blocks, a sequence-based compact model generates 2^k distinct mathematical vectors in the variable space that evaluate to identical or mathematically symmetric physical transition states, creating an exponentially large degenerate plateau in the branch-and-bound tree.*

Proof. Let $B_i \in \mathcal{B}$ be a 0-block for tool t_i spanning the discrete positional interval $[p, q]$. The tool is strictly required at position $p - 1$ and position $q + 1$, meaning $a^{\sigma}_{t_i,p-1} = 1$ and $a^{\sigma}_{t_i,q+1} = 1$, while $a^{\sigma}_{t_i,r} = 0$ for all $r \in [p, q]$.

In a compact model, the state of the magazine across this interval is dictated by the vector of binary presence variables $(v_{p,t_i}, v_{p+1,t_i}, \ldots, v_{q,t_i})$. Because the tool is not strictly required by the incidence matrix during this interval, the coverage constraint $v_{r,t_i} \geq a^{\sigma}_{t_i,r}$ simply evaluates to $v_{r,t_i} \geq 0$, rendering it non-binding.

The solver faces a binary choice: either retain the tool across the entire block (flipping the 0-block by setting $v_{r,t_i} = 1$) or evict it and reload it later. For a set of k independent 0-blocks, the solver treats each interval retention as a decoupled binary decision $\delta_i \in \{0, 1\}$ for $i \in \{1, \ldots, k\}$. The Cartesian product of these independent binary decisions yields exactly $|\{0, 1\}^k| = 2^k$ unique assignment permutations in the $\mathbf{v}$ variable space.

If the objective function explicitly penalized every eviction, the solver would immediately gravitate toward the single permutation where all $\delta_i = 1$. However, the objective function exclusively penalizes *insertions* ($w_{r,t} = \max\{0, v_{r,t} - v_{r-1,t}\}$). Consider a sub-permutation where a tool t_i is evicted at position p ($\delta_i = 0$) and re-inserted at position $q + 1$. This incurs exactly one insertion penalty. Now consider an alternative permutation where t_i is arbitrarily retained until position $p+1$ and then evicted, or evicted at p but arbitrarily re-inserted at q prior to its actual requirement.

Because the capacity bound b is not tight (by the definition of independent 0-blocks), the solver can shift the exact position of the empty slots

seamlessly. The structural transition cost remains exactly identical (e.g., 1 penalty), but the specific mathematical vector $(v_{p,t_i}, \ldots, v_{q,t_i})$ changes.

Consequently, the branch-and-bound tree must explicitly generate, evaluate, and symmetrically prune up to 2^k equivalent matrix configurations. Even if the bounds are partially tightened by valid inequalities, the solver is fundamentally forced to distinguish between identical physical machine states merely because the arbitrary presence of unrequired, zero-cost dummy tools can be permuted in 2^k distinct ways across the positional variables. □

2.3.3 Theorem and Proof of LP Lower Bound Collapse

Building upon the fractional assignment analysis and the combinatorial explosion of 0-block symmetries, we can now formally establish the fundamental mathematical failure of the linear relaxation for sequence-based compact models. The core operational metric of the SSP—sequence-dependent setup costs—is entirely nullified in the continuous polyhedral space.

Theorem 2.3 (LP Lower Bound Collapse)**.** *Let Z_{LP} denote the optimal objective value of the continuous linear relaxation of the position-based compact formulation for a U-SSP instance. If the global tool requirements satisfy the uniform fractional distribution condition $\frac{1}{n}\sum_{j=1}^{n} a_{tj} \leq \frac{b}{m}$ for all $t \in T$, then the continuous lower bound collapses to exactly $Z_{LP} = 0$, completely independent of the true integer optimal cost Z_{IP}.*

Proof. Consider the linear relaxation of the position-based model where the binary assignment and state constraints are relaxed to the continuous domain: $u_{jk}, v_{kt}, w_{kt} \in [1]$.

We construct a completely uniform, sequence-independent fractional solution. Let the assignment variables be distributed uniformly across all discrete positions: $u_{jk} = \frac{1}{n}$ for all $j \in J$ and $k \in K$. This static distribution strictly satisfies the doubly stochastic assignment constraints:

$$\sum_{k=1}^{n} u_{jk} = \sum_{k=1}^{n} \frac{1}{n} = 1 \quad \forall j \in J, \qquad \sum_{j=1}^{n} u_{jk} = \sum_{j=1}^{n} \frac{1}{n} = 1 \quad \forall k \in K. \tag{2.31}$$

To satisfy the required tool coverage constraint $v_{kt} \geq \sum_j a_{tj} u_{jk}$, we set the continuous magazine state variables exactly to their fractional requirement bound. Since $u_{jk} = \frac{1}{n}$ globally, the state variable becomes entirely independent of the positional index k:

$$v_{kt} = \frac{1}{n} \sum_{j=1}^{n} a_{tj} \quad \forall k \in K, \forall t \in T. \tag{2.32}$$

We must mathematically verify that this static fractional magazine state respects the physical capacity constraint $\sum_t v_{kt} \leq b$ at all times. By sum-

ming the fractional presence over the entire tool set T:

$$\sum_{t \in T} v_{kt} = \sum_{t \in T} \left(\frac{1}{n} \sum_{j=1}^{n} a_{tj} \right) = \frac{1}{n} \sum_{j=1}^{n} \left(\sum_{t \in T} a_{tj} \right). \tag{2.33}$$

By the foundational definition of the SSP, no individual job requires a subset of tools exceeding the magazine capacity, meaning $\sum_{t \in T} a_{tj} = |T_j| \leq b$ for all $j \in J$. Substituting this absolute bound yields:

$$\sum_{t \in T} v_{kt} \leq \frac{1}{n} \sum_{j=1}^{n} b = \frac{1}{n}(n \cdot b) = b. \tag{2.34}$$

Thus, the aggregate fractional presence never violates the capacity constraint at any position k, seamlessly bypassing the strict physical bottlenecks of the discrete machine.

Finally, we evaluate the objective function, which minimizes the summation of transition variables subject to the gradient bound $w_{kt} \geq v_{kt} - v_{k-1,t}$. Because our constructed fractional state v_{kt} is strictly uniform across all positions k, the temporal state gradient evaluates exactly to zero:

$$v_{kt} - v_{k-1,t} = \left(\frac{1}{n} \sum_{j=1}^{n} a_{tj} \right) - \left(\frac{1}{n} \sum_{j=1}^{n} a_{tj} \right) = 0 \quad \forall k \in K, \forall t \in T. \tag{2.35}$$

Because the objective acts to strictly minimize the non-negative variables w_{kt}, the optimal continuous resolution bounds the insertion penalties to exactly $w_{kt} = 0$. Consequently, the global objective function equates to the trivial floor $Z_{LP} = \sum_k \sum_t w_{kt} = 0$.

This explicitly proves that the LP solver effortlessly accommodates all discrete scheduling constraints by exploiting the fractional domain, mathematically "smearing" the tooling requirements homogeneously across the temporal horizon to achieve a zero-cost transition matrix. The resulting theoretical lower bound $Z_{LP} = 0$ offers absolutely no valid combinatorial insight. □

The mathematical consequences of Theorem 2.2 and the 2^k permutational symmetry (Theorem 2.1) are computationally devastating [**?**, **?**]. When an exact Mixed-Integer Linear Programming (MILP) solver attempts to resolve these sequence-based compact models, the root node yields a weak, heavily fractionalized lower bound. As the branch-and-bound engine begins fixing variables into the binary domain, it inevitably collides with the exponentially massive plateau of symmetric 0-block permutations. Because traversing these arbitrary retention pathways yields no immediate improvement in the fractional bound, the solver suffers from extreme dual degeneracy, stalling infinitely on moderately sized industrial instances.

It is this specific polyhedral failure that necessitates a radical paradigm shift in the algorithmic approach to the U-SSP. To mathematically escape the "Symmetry Menace" and circumvent the LP lower bound collapse, the formulation architecture must abstract away from relative positional routing variables. Instead, the problem must be mapped directly onto the power set of maximal feasible tool configurations. This fundamental transition from a sequential space to a set-covering space lays the rigorous theoretical foundation for the Job Grouping Problem (JGP) and the advanced Dantzig-Wolfe column generation frameworks explored in the subsequent chapters.

Chapter 3

Topological Shifts: The Non-Compact Configuration Space

3.1 Defining the Configuration Graph Domain

The insurmountable computational bottlenecks of sequence-based compact formulations—specifically, the 2^k permutational symmetry of 0-blocks and the subsequent collapse of the linear relaxation lower bound—necessitate a fundamental paradigm shift in the mathematical modeling of the Uniform Job Sequencing and Tool Switching Problem (U-SSP). To eliminate the fractional degeneracy associated with localized temporal tracking, advanced optimization frameworks abstract the state space entirely away from positional assignment variables. Instead, the combinatorial architecture of the problem is mapped directly onto the discrete power set of all feasible machine states.

3.1.1 The Global Configuration Space and Job Clusters

We abandon the discrete positional transition variables and explicitly define the physical state of the tool magazine as an independent polyhedral entity. Building upon the established global optimality of dummy tool padding, we restrict our focus exclusively to maximal configurations, where the magazine is strictly saturated to its physical limit at all times.

Let $\mathcal{C}$ denote the global configuration space, defined rigorously as the set of all possible maximal tool configurations that can be constructed from the available tool set T:

$$\mathcal{C} = \{C \subseteq T \mid |C| = b\}. \tag{3.1}$$

The exact cardinality of this non-compact configuration space is dictated by

the binomial coefficient:

$$|\mathcal{C}| = \binom{m}{b}. \tag{3.2}$$

For any industrial-sized instance where the tool library m and magazine capacity b are sufficiently large, the sheer exponential scale of $|\mathcal{C}|$ renders explicit algorithmic enumeration intractable. However, projecting the problem into this massive space provides the exact polyhedral geometry required to formulate a theoretically tight linear relaxation, entirely bypassing the temporal symmetries of the position-based models.

Rather than evaluating the fractional presence of an individual tool t at a temporal step k, this framework evaluates whether an entire, structurally complete configuration is capable of processing a given job. We formally define the *configuration cluster* H_j as the specific subset of all maximal configurations in $\mathcal{C}$ that contain the requisite tool set T_j for job $j \in J$:

$$H_j = \{C \in \mathcal{C} \mid T_j \subseteq C\}. \tag{3.3}$$

Because it is physically mandated that $|T_j| \leq b$ for all $j \in J$, the subset H_j is mathematically guaranteed to be non-empty for every valid job. Geometrically, H_j acts as a localized feasibility region within the broader combinatorial space $\mathcal{C}$.

By adopting this non-compact, set-covering perspective, the fundamental objective of the U-SSP undergoes a critical topological translation. The problem is no longer defined as sorting a one-dimensional permutation $\sigma \in S_n$. Instead, the U-SSP is rigorously reformulated as the search for a minimum-cost directed path through the configuration graph $\mathcal{C}$, subject to the constraint that the optimal path must intersect at least one configuration node within every job cluster H_j for all $j \in J$.

In practical operational environments, a machine might process a job using a subset of tools strictly smaller than the magazine's capacity. However, any such under-capacity state is structurally dominated by a maximal state by Lemma 1.2. Hence, we justify the restriction of the global configuration space $\mathcal{C}$ exclusively to maximal configurations (where $|C| = b$).

3.2 Transition Metrics and Polyhedral Geometry

3.2.1 Derivation of the Transition Cost Metric

The fundamental topology of the non-compact formulation is governed by the distance metric that maps the transition costs between valid states in the configuration graph. In the context of the Uniform Job Sequencing and Tool Switching Problem (U-SSP), the underlying operational assumption strictly enforces that every tool requires an identical mechanical effort to change [**?**]. Consequently, the distance metric between any two configurations must

accurately quantify the minimum number of discrete mechanical operations required to physically transform the CNC machine's tool magazine from an initial state to a target state [**?**].

Let $C_u, C_v \in \mathcal{C}$ represent two arbitrary maximal configurations within the global state space. By the definition of the maximal configuration space $\mathcal{C}$, both sets strictly saturate the machine's magazine capacity, mathematically enforcing the equality $|C_u| = |C_v| = b$ [**?**].

When the machine transitions from processing a job utilizing configuration C_u to a subsequent job requiring configuration C_v, the automated tool changer must extract unnecessary tools to free capacity for the newly required ones [**?**]. For any two configurations C_u, C_v, the transition cost $d(C_u, C_v)$ is defined precisely by the number of tool switches required to move from state C_u to state C_v [**?**]. The specific set of tools that naturally persists across this transition without requiring mechanical handling is defined by the intersection $C_u \cap C_v$ [**?**].

To accommodate the new tools required by C_v that are absent in C_u, the machine must evict a corresponding subset of tools [**?**]. Because the magazine capacity b is strictly saturated, the exact number of tool insertions—which defines the operational transition cost—is equal to the capacity minus the cardinality of the shared intersection [**?**]. We can therefore formally define the uniform continuous transition cost metric $d : \mathcal{C} \times \mathcal{C} \to \mathbb{Z}^+ \cup \{0\}$ as:

$$d(C_u, C_v) = b - |C_u \cap C_v| \tag{3.4}$$

This algebraically rigorous definition explicitly binds the physical realities of the CNC machinery to the abstract combinatorial graph space. By representing the setup penalty as a continuous, set-theoretic distance evaluated across static configuration vertices, the non-compact model successfully establishes a geometric routing space entirely shielded from the fractional degeneracies that collapse traditional compact models.

3.2.2 Theorem and Proof: Metric Space Validity

For the non-compact configuration graph $\mathcal{C}$ to be rigorously transformed into a classical routing domain, such as the Generalized Traveling Salesman Problem (GTSP), its transition metric must strictly satisfy the topological properties of a metric space [**?**]. While non-negativity and symmetry are trivially satisfied by the algebraic definition $d(C_u, C_v) = b - |C_u \cap C_v|$, establishing the triangle inequality requires a formal set-theoretic proof [**?**].

Theorem 3.1 (Triangle Inequality of Configuration Transitions)**.** *For any three arbitrary maximal configurations $C_u, C_w, C_v \in \mathcal{C}$, the transition metric satisfies the triangle inequality:*

$$d(C_u, C_v) \leq d(C_u, C_w) + d(C_w, C_v). \tag{3.5}$$

Proof. By the operational definition derived in Section 3.2.1, the distance $d(C_u, C_v)$ is fundamentally equivalent to the number of tools that must be mechanically evicted from the magazine when transitioning directly from state C_u to state C_v [**?**]. Mathematically, this eviction set is precisely defined by the relative complement, allowing us to rewrite the distance metric via cardinality:

$$d(C_u, C_v) = |C_u \setminus C_v|. \tag{3.6}$$

Consider an indirect routing sequence transitioning through an intermediate configuration C_w. For any specific tool $t \in T$ that must be mechanically removed when transitioning directly from C_u to C_v, we have by definition that $t \in C_u$ and $t \notin C_v$.

When evaluating the presence of this specific tool t within the intermediate state C_w, the Law of Excluded Middle dictates exactly two exhaustive logical possibilities: either $t \in C_w$ or $t \notin C_w$.

- **Case 1 ($t \notin C_w$):** If the tool is absent in the intermediate configuration C_w, the CNC machine must have mechanically evicted it during the initial transition from C_u to C_w. Therefore, we have $t \in (C_u \setminus C_w)$.

- **Case 2 ($t \in C_w$):** If the tool is present in the intermediate configuration C_w, but absent in the target configuration C_v, the machine must evict it during the secondary transition from C_w to C_v. Therefore, we have $t \in (C_w \setminus C_v)$.

Because every single evicted tool $t \in (C_u \setminus C_v)$ must strictly satisfy at least one of these two intermediate conditions, the relative complement defining the direct transition is strictly a subset of the union of the relative complements of the intermediate transitions:

$$C_u \setminus C_v \subseteq (C_u \setminus C_w) \cup (C_w \setminus C_v). \tag{3.7}$$

Applying the cardinality operator to both sides of this subset relation, and invoking the fundamental subadditivity property of the set union operator, we establish the following bounding inequality [**?**]:

$$|C_u \setminus C_v| \leq |(C_u \setminus C_w) \cup (C_w \setminus C_v)| \leq |C_u \setminus C_w| + |C_w \setminus C_v|. \tag{3.8}$$

Substituting the relative complement distance equivalence $d(A, B) = |A \setminus B|$ back into this cardinality bound yields the final theorem statement [**?**]:

$$d(C_u, C_v) \leq d(C_u, C_w) + d(C_w, C_v). \tag{3.9}$$

□

The mathematical consequences of Theorem 3.2 are profound. Because the distance function $d(C_u, C_v)$ evaluates to a non-negative symmetric integer and obeys the triangle inequality, the combinatorial configuration graph $\mathcal{C}$ constitutes a strictly valid metric space [**?**]. This geometric property acts as the foundational prerequisite allowing us to extract the sequencing logic from the temporal domain and map it safely into established structural algorithms designed for metric Traveling Salesman and Vehicle Routing networks.

3.3 The Non-Compact Integer Linear Programming Formulation

3.3.1 Variables, Objective Function, and Base Constraints

Having established that the configuration graph $\mathcal{C}$ constitutes a valid metric space, the Tool Switching Problem can be rigorously projected into this topology. The non-compact formulation models the SSP as the search for a minimum-cost directed path through configuration vertices that effectively covers all job requirements [**?**].

Let $H_j \subseteq \mathcal{C}$ denote the cluster of valid maximal configurations capable of processing job $j \in J$, defined precisely as $H_j = \{C \in \mathcal{C} \mid T_j \subseteq C\}$ [**?**]. To map the combinatorial routing decisions, we introduce two primary sets of binary decision variables defined over the continuous configuration space:

- $y(v) \in \{0, 1\}$: A binary activation variable equal to 1 if the specific maximal configuration $v \in \mathcal{C}$ is selected to process at least one job in the sequence, and 0 otherwise.
- $x(u, v) \in \{0, 1\}$: A binary routing variable equal to 1 if the machine transitions directly from configuration $u \in \mathcal{C}$ to configuration $v \in \mathcal{C}$, and 0 otherwise.

The fundamental objective is to minimize the total mechanical transition costs accumulated across the selected configuration path.

$$\text{Minimize} \quad Z = \sum_{u \in \mathcal{C}} \sum_{v \in \mathcal{C}} d(u, v) x(u, v) \tag{3.10}$$

The feasibility of the processing sequence is strictly dictated by the structural coverage and network flow conservation constraints:

$$\sum_{v \in H_j} y(v) \geq 1 \qquad \forall j \in J \tag{3.11}$$

$$\sum_{u \in \mathcal{C}} x(u, v) = y(v) \qquad \forall v \in \mathcal{C} \tag{3.12}$$

$$\sum_{u \in \mathcal{C}} x(v, u) = y(v) \qquad \forall v \in \mathcal{C} \tag{3.13}$$

Constraint (3.11) represents the generalized set-covering condition. It mathematically mandates that for every job $j \in J$, the final sequence must activate at least one configuration vertex belonging to its feasibility cluster H_j, guaranteeing that all jobs are successfully processed [**?**].

Constraints (3.12) and (3.13) govern the nodal flow conservation within the configuration graph. They rigorously enforce that if a configuration v is activated ($y(v) = 1$), it must possess exactly one incoming routing edge and exactly one outgoing routing edge, securing a continuous operational sequence [**?**]. Conversely, if the configuration is not selected ($y(v) = 0$), it is mathematically isolated from the transition path, strictly carrying zero flow.

3.3.2 Generalized Subtour Elimination and Overlapping Cluster Validity

To strictly prevent the formulation from selecting isolated, disconnected cycles of tool configurations that fail to form a single, continuous sequence through the required jobs, we must introduce a family of Generalized Subtour Elimination Constraints (GSECs) [**?**].

For any proper subset of jobs $S \subset J$ such that $2 \leq |S| \leq |J| - 1$, we define the aggregated configuration subset $H(S)$ as the strict union of the individual job clusters: $H(S) = \bigcup_{j \in S} H_j$ [**?**]. The corresponding polyhedral inequality is formulated as follows:

$$\sum_{u \in H(S)} \sum_{v \in H(S)} x(u, v) \leq \sum_{v \in H(S)} y(v) - 1 \quad \forall S \subset J, 2 \leq |S| \leq |J| - 1 \tag{3.14}$$

A critical mathematical nuance of this non-compact formulation is the theoretical validity of these GSECs in the presence of heavily overlapping configuration clusters [**?**]. In standard Generalized Traveling Salesman Problem (GTSP) models, the vertices are partitioned into mutually exclusive, strictly disjoint sets [**?**]. However, in the non-compact SSP graph, a single maximal tool configuration can easily satisfy the requirements of multiple jobs simultaneously, mathematically implying that $H_i \cap H_k \neq \emptyset$ [**?**].

Despite this structural overlap, the topological logic of inequality (3.14) remains entirely valid [**?**]. The constraint operates strictly over the unique configuration vertices activated to serve a specific job subset [**?**]. If an isolated sub-cycle were to form exclusively among the configurations in $H(S)$, the number of internal transition edges $\sum x(u, v)$ would perfectly equal the number of unique active configurations $\sum y(v)$ within that sub-cycle [**?**]. By bounding the sum of these internal routing edges strictly below the absolute number of unique activated vertices in that spatial region, the formulation correctly forbids both integer and fractional sub-cycles from achieving feasibility [**?**]. This mathematical logic holds perfectly regardless of how many

individual jobs a single configuration vertex might cover, successfully preserving the integrity of the routing sequence [**?**].

Chapter 4

Structural Equivalences to Generalized Routing Problems

4.1 The SSP to GTSP Forward Mapping

4.1.1 The Overlap Problem and Topological Mismatch

To construct an exact algorithmic resolution for the Uniform Job Sequencing and Tool Switching Problem (U-SSP) leveraging the non-compact configuration graph $\mathcal{C}$, we must mathematically project the problem into an established combinatorial routing framework. The most natural topological candidate is the Generalized Traveling Salesman Problem (GTSP). In the standard GTSP literature, the routing topology is rigorously defined over a graph where the vertex set is partitioned into a family of mutually exclusive, strictly disjoint clusters , and the objective is to find a minimum-cost cycle that visits exactly one vertex from each cluster.

While the non-compact formulation of the SSP visually resembles the GTSP—requiring the selection of one feasible configuration vertex from each job cluster H_j—a fundamental topological mismatch precludes a direct isomorphic mapping. This discrepancy is formally defined as the *Overlap Problem*.

In the U-SSP, the configuration clusters are not mutually exclusive. A single maximal tool configuration $C \in \mathcal{C}$ might simultaneously satisfy the tool requirements of multiple distinct jobs. If a configuration C contains the union of the tool requirements for both job i and job k (i.e., $(T_i \cup T_k) \subseteq C$), then by definition, $C \in H_i$ and $C \in H_k$. Consequently, the geometric intersection of the two clusters is strictly non-empty:

$$H_i \cap H_k \neq \emptyset. \tag{4.1}$$

Because a standard GTSP solver inherently assumes that the selection of a vertex satisfies exactly one cluster, injecting overlapping sets directly into a GTSP algorithm will induce severe constraint violations or erroneous routing evaluations. The mathematical structure must therefore be transformed to synthesize artificial disjointness without corrupting the underlying transition metrics.

4.1.2 The Intersecting-to-Nonintersecting (I-N) Transformation

To resolve the structural overlap and achieve a perfect mathematical equivalence to the GTSP, we apply an adaptation of the Intersecting-to-Nonintersecting (I-N) graph transformation, originally proposed for overlapping routing domains . The foundational logic of this transformation relies on systematic vertex duplication to explode the intersecting configuration space into parallel, job-specific sub-spaces.

Let the original U-SSP instance be defined by the tuple $\mathcal{I}_{SSP} = \langle J, T, b, \{T_j\}_{j \in J} \rangle$. We construct a corresponding, strictly disjoint GTSP instance $\mathcal{I}_{GTSP} = \langle \mathcal{V}^*, \mathcal{E}^*, d^*, \{V_j^*\}_{j \in J} \rangle$ through the following algorithmic projection:

1. **Vertex Replication and Disjoint Partitioning:** For every individual job $j \in J$, we instantiate a unique, isolated cluster V_j^*. This cluster is populated exclusively with job-specific replicas of all maximal configurations in the global space $\mathcal{C}$ capable of processing job j. We formally define the replica set as:

$$V_j^* = \{v_{C,j} \mid C \in \mathcal{C},\ T_j \subseteq C\} \quad \forall j \in J. \tag{4.2}$$

   Because every replica vertex $v_{C,j}$ is explicitly indexed by its target job j, a configuration C that satisfies both jobs i and k is physically bifurcated into two entirely distinct graph nodes: $v_{C,i} \in V_i^*$ and $v_{C,k} \in V_k^*$. This replication guarantees that the resulting cluster partition is absolutely mutually exclusive:

$$V_i^* \cap V_k^* = \emptyset \quad \forall i, k \in J,\ i \neq k. \tag{4.3}$$

2. **Transition Metric Preservation:** The distance function d^* defining the edge weights $\mathcal{E}^*$ in the duplicated graph must perfectly preserve the mechanical realities of the CNC tool magazine. For any two replica vertices residing in completely distinct job clusters, the transition cost is inherited directly from the set-theoretic distance of their origin configurations:

$$d^*(v_{C_u,i}, v_{C_w,k}) = b - |C_u \cap C_w| \quad \forall i \neq k. \tag{4.4}$$

3. **Zero-Cost Intra-Configuration Routing:** The critical mathematical elegance of the I-N transformation emerges when evaluating edges between replicas of the *exact same* configuration across different job clusters. If the machine sequentially processes job i and job k utilizing the exact same physical magazine setup C, the transition cost must be zero. By applying our derived metric to two replicas of the identical origin configuration C, the intersection is absolute:

$$d^*(v_{C,i}, v_{C,k}) = b - |C \cap C| = b - b = 0. \tag{4.5}$$

By structurally separating the vertices but linking identical configurations with zero-cost temporal bridges, the I-N transformation successfully maps the U-SSP into a valid, disjoint GTSP network.

4.1.3 Visualizing the State Space Split

The structural mechanics of this duplication, illustrating how an intersecting configuration is mathematically bifurcated into distinct routing nodes, is visualized in Figure 4.1.

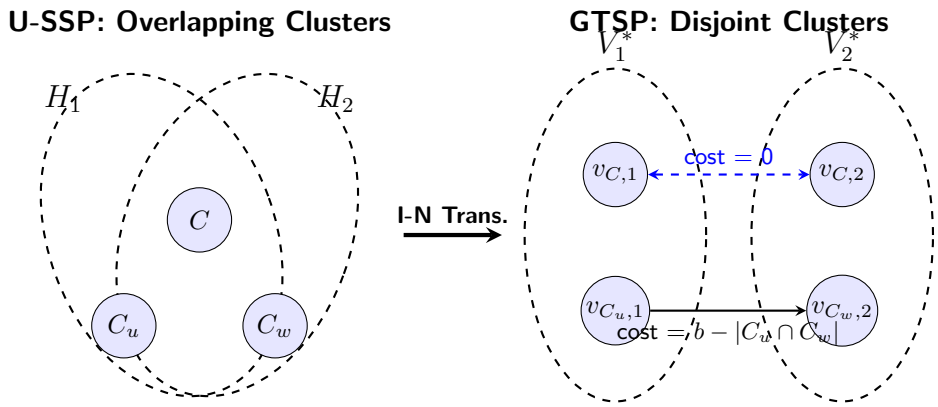

Figure 4.1: The Intersecting-to-Nonintersecting (I-N) transformation. A single maximal configuration C capable of processing both Job 1 and Job 2 naturally resides in the mathematical intersection of clusters H_1 and H_2. To establish routing disjointness, the algorithm explicitly duplicates this configuration into job-specific replicas $v_{C,1} \in V_1^*$ and $v_{C,2} \in V_2^*$. The topological distance between these distinct vertices evaluates strictly to zero, preserving the underlying physical mechanics.

4.1.4 Theorem and Proof: Bijection and Optimality Preservation

To definitively validate the topological shift from the non-compact SSP to the GTSP routing domain, we must mathematically prove that the projec-

tion into an artificially duplicated, disjoint state space incurs strictly zero loss of global optimality [**?**].

Theorem 4.1 (SSP-GTSP Bijection)**.** *An optimal, minimum-cost Hamiltonian cycle in the constructed GTSP instance $\mathcal{I}_{GTSP}$ corresponds precisely to an optimal sequence of configurations in the original U-SSP instance $\mathcal{I}_{SSP}$, with mathematically identical switching costs and no loss of optimality.*

Proof. Let Z^*_{GTSP} be a geometrically feasible, minimum-cost Hamiltonian cycle in the constructed graph $\mathcal{I}_{GTSP}$. By the fundamental constraint definition of the GTSP, this cyclic sequence is strictly mandated to visit exactly one replica vertex from each disjoint job cluster V^*_j for all $j \in J$.

In the physical operational context of the U-SSP, selecting exactly one replica $v_{C,j} \in V^*_j$ structurally maps to selecting exactly one valid, maximal magazine configuration $C \in H_j$ to successfully process job j, thereby establishing a continuous processing sequence.

Consider a specific configuration C that is capable of satisfying a contiguous sub-sequence of jobs, say $j_1, j_2, \ldots, j_k$. In the original discrete SSP space, the optimal Keep Tool Needed Soonest (KTNS) policy natively traverses this 0-block by simply retaining the physical magazine state C across the entire temporal interval, incurring zero mechanical setup penalties.

In the transformed GTSP graph, the corresponding mathematical replicas of this shared configuration are distributed strictly across their respective disjoint clusters: $v_{C,j_1} \in V^*_{j_1}$, $v_{C,j_2} \in V^*_{j_2}$, and so forth. Because the I-N algorithm explicitly connects these identical replicas with dedicated inter-cluster edges possessing an assigned cost of zero, the GTSP routing model is permitted to seamlessly transition between these nodes.

By deliberately traversing these zero-cost edges, the GTSP cycle effectively "dwells" on the single, underlying physical machine state C while mathematically satisfying multiple, consecutive job requirements. This zero-cost topological dwelling acts as a perfect mathematical emulation of the KTNS policy traversing a continuous 0-block.

For any transition between fundamentally distinct physical configurations C_u and C_w, the corresponding edge weight $d^*(v_{C_u,i}, v_{C_w,k})$ is uniformly evaluated exactly as $b - |C_u \cap C_w|$. This perfectly preserves the discrete unit cost of every individual tool eviction and insertion operation required by the machine.

Consequently, any non-redundant sequence of disjoint clusters in the GTSP graph collapses directly into a valid, sequence-preserving tool-loading policy for the SSP. Because the local set-theoretic transition costs and the zero-cost idle blocks are perfectly equivalent, the sum of the edge weights on the GTSP cycle is exactly equal to the total number of physical tool switches required by the corresponding SSP sequence. Therefore, minimizing the cycle length strictly yields the minimal switching cost, establishing an absolute structural bijection. □

4.2 Topological Thresholds: The Disjointness Condition

4.2.1 Theorem and Proof of the Disjointness Condition

While the Intersecting-to-Nonintersecting (I-N) transformation successfully achieves a mathematically rigorous mapping from the Uniform Job Sequencing and Tool Switching Problem (U-SSP) to the Generalized Traveling Salesman Problem (GTSP), it incurs a severe computational penalty. By explicitly duplicating intersecting configuration vertices across all applicable job clusters, the I-N algorithm drastically inflates the dimensionality of the routing graph .

However, the necessity of this exponential duplication is strictly contingent upon the actual presence of overlapping clusters within the configuration space. We propose that the topological architecture of the SSP natively subsumes the GTSP as a specific boundary case. To establish the precise threshold where the U-SSP collapses into a pure, disjoint GTSP without requiring any artificial vertex replication, we introduce the Disjointness Condition.

Theorem 4.2 (The Disjointness Condition)**.** *Let $\mathcal{I}_{SSP} = \langle J, T, b, \{T_j\}_{j \in J} \rangle$ be a U-SSP instance. For any two distinct jobs $i, k \in J$ with respective tool requirements T_i and T_k, if the strict cardinality of their set union exceeds the physical magazine capacity, meaning $|T_i \cup T_k| > b$, then their corresponding configuration clusters H_i and H_k are natively mutually exclusive, establishing that $H_i \cap H_k = \emptyset$.*

Proof. We proceed via a formal proof by contradiction.

Assume the premise holds, meaning the collective tool requirement for jobs i and k strictly exceeds the available magazine slots:

$$|T_i \cup T_k| > b. \tag{4.6}$$

Suppose, for the sake of contradiction, that the configuration clusters are not mutually exclusive. Therefore, their geometric intersection is strictly non-empty:

$$H_i \cap H_k \neq \emptyset. \tag{4.7}$$

By the definition of a non-empty intersection, there must exist at least one valid maximal configuration $C \in \mathcal{C}$ that resides within both localized feasibility regions. Thus, $C \in H_i$ and simultaneously $C \in H_k$.

By the fundamental set-theoretic definition of a configuration cluster ($H_j = \{C \in \mathcal{C} \mid T_j \subseteq C\}$), membership in these respective clusters imposes the following strict subset relations:

$$C \in H_i \implies T_i \subseteq C, \tag{4.8}$$

$$C \in H_k \implies T_k \subseteq C. \tag{4.9}$$

If a set C acts as a superset to both T_i and T_k, fundamental set theory mandates that C must also act as a superset to their mathematical union:

$$(T_i \cup T_k) \subseteq C. \tag{4.10}$$

Applying the cardinality operator to both sides of this subset relation rigorously preserves the bounding inequality:

$$|T_i \cup T_k| \leq |C|. \tag{4.11}$$

By the definitional axioms of the global configuration space $\mathcal{C}$, any valid configuration C must strictly respect the physical hardware limits of the CNC machine's tool magazine. Because we restrict the space exclusively to maximal configurations, the cardinality of C evaluates to exactly the capacity limit:

$$|C| = b. \tag{4.12}$$

Substituting equation (4.12) into inequality (4.11) yields the following topological certainty for any configuration residing in the intersection:

$$|T_i \cup T_k| \leq b. \tag{4.13}$$

This mathematical deduction constitutes an absolute, irreconcilable contradiction with our initial premise (4.6), which established that $|T_i \cup T_k| > b$. Because the existence of any intersecting configuration mathematically forces a violation of the machine's strict physical capacity, no such configuration can exist.

Therefore, the initial assumption that the intersection is non-empty must be false, conclusively proving that $H_i \cap H_k = \emptyset$. □

The complexity implications of Theorem 4.2 are profound for combinatorial operations research . If a specific industrial instance of the U-SSP features highly heterogeneous, specialized jobs such that $|T_i \cup T_k| > b$ evaluates to true for every single pair of distinct jobs $i \neq k$, then the entire family of configuration clusters $\{H_1, \ldots, H_n\}$ is natively mutually exclusive .

Under these specific conditions, the Overlap Problem vanishes entirely. The U-SSP completely bypasses the need for the exponential I-N vertex duplication algorithm and maps directly, with perfect isomorphism, onto the classical Generalized Traveling Salesman Problem . This establishes a formal structural subsumption: the standard GTSP is not merely an analogous routing domain, but rather a strict, mathematically bounded special case of the non-compact Tool Switching Problem where the intersection of feasibility clusters is natively empty .

4.3 Reverse Equivalence Mapping (Restricted GTSP to SSP)

4.3.1 Restricted GTSP Boundaries and the Metric Barrier

While the Intersecting-to-Nonintersecting (I-N) transformation successfully establishes a formal, polynomial forward reduction from the Uniform Job Sequencing and Tool Switching Problem (U-SSP) to the Generalized Traveling Salesman Problem (GTSP) , establishing a generalized reverse reduction from an arbitrary GTSP back to the U-SSP is mathematically prohibited. This structural limitation is formally defined in the combinatorial routing literature as the *Metric Barrier*.

In a standard Generalized Traveling Salesman Problem, the topological routing domain is defined over a complete graph where the transition edge costs between vertices are permitted to be arbitrary, real-valued, and highly asymmetric . Conversely, the fundamental topological architecture of the U-SSP is governed strictly by the mechanical and physical realities of the CNC tool magazine.

Under the uniform setup assumption, the transition metric evaluating the discrete distance between any two maximal configurations C_u and C_v is rigorously evaluated as $d(C_u, C_v) = b - |C_u \cap C_v|$. This specialized, set-theoretic distance function imposes an extremely rigid geometry on the valid state space.

Theorem 4.3 (The Metric Barrier)**.** *A general Generalized Traveling Salesman Problem (GTSP) instance $\mathcal{I}_{GTSP} = \langle \mathcal{V}, \mathcal{E}, w, \{V_j\}_{j \in J} \rangle$ cannot be polynomially reduced to a Uniform Tool Switching Problem (U-SSP) unless its edge weight function $w : \mathcal{E} \to \mathbb{R}$ strictly conforms to the U-SSP polyhedral boundaries.*

Proof. Let $\mathcal{C}$ be the universal configuration space for a U-SSP instance with magazine capacity b. The mechanical transition cost function $d : \mathcal{C} \times \mathcal{C} \to \mathbb{Z}^+ \cup \{0\}$ inherently exhibits the following four absolute topological properties:

1. **Symmetry:** Because the intersection operator is fundamentally commutative ($C_u \cap C_v = C_v \cap C_u$), the distance is strictly symmetric: $d(C_u, C_v) = d(C_v, C_u)$ for all valid configurations .

2. **Integrality:** The cardinality of any set intersection is an integer, mathematically forcing $d(C_u, C_v) \in \mathbb{Z}^+ \cup \{0\}$.

3. **Capacity Bounding:** Because the intersection cardinality is non-negative and naturally bounded by the set sizes ($0 \leq |C_u \cap C_v| \leq b$), the maximum possible transition cost is strictly bounded by the hardware capacity: $d(C_u, C_v) \leq b$.

4. **Metric Space Validity:** As proven previously in Theorem 3.2, the distance function inherently satisfies the continuous triangle inequality $d(C_u, C_v) \leq d(C_u, C_w) + d(C_w, C_v)$.

Suppose there exists an arbitrary GTSP instance containing a directed edge $(u, v) \in \mathcal{E}$ with an asymmetric weight $w(u, v) \neq w(v, u)$, a fractional weight $w(u, v) = 2.5$, or a weight strictly exceeding the maximum permissible magazine capacity $w(u, v) > b$.

It is mathematically impossible to construct a pair of physical tool configurations C_u and C_v whose set-theoretic intersection yields a cardinality that simultaneously satisfies the equality $b - |C_u \cap C_v| = w(u, v)$ under these arbitrary conditions.

Because the general GTSP permits edge weights that freely violate symmetry, integrality, hardware capacity bounds, and the triangle inequality , no generalized polynomial mapping can exist that translates these arbitrary routing matrices back into the rigid intersection mechanics of the U-SSP. □

Consequently, only a highly restricted sub-class of symmetric GTSP instances—specifically those where the vertex clusters and integer edge weights perfectly mirror the intersection cardinality of discrete b-bounded sets—can be structurally reverse-mapped into the tool switching operational environment.

4.3.2 Formalization of Dummy Tool Padding

To definitively bridge the topological gap between the Uniform Job Sequencing and Tool Switching Problem (U-SSP) and the restricted Generalized Traveling Salesman Problem (GTSP), the state space must universally enforce the symmetric intersection metric $d(C_u, C_v) = b - |C_u \cap C_v|$ across all routing edges. If the solver evaluates a configuration that is strictly under-capacity ($|C_u| < b$), the transition metric loses its structural symmetry and polynomial equivalence to fixed-capacity GTSP edge weights.

We resolve this geometry via the formalization of *Dummy Tool Padding*, mathematically engineering the U-SSP transition costs to emulate specific, bounded integer weights without degrading the objective function.

Theorem 4.4 (Configuration Maximization for Metric Symmetry)**.** *Let $\mathcal{I}_{SSP}$ be a uniform tool switching instance with capacity b. Any dynamically generated configuration C_u satisfying a job $u \in J$ such that $|C_u| < b$ can be mapped to a maximal configuration $C'_u = C_u \cup T_{dummy}$, where $|C'_u| = b$ and $T_{dummy} \subseteq T \setminus C_u$. This mapping preserves the symmetric metric barrier requirements of the GTSP while ensuring the outbound transition cost is strictly monotonically non-increasing: $d(C'_u, C_v) \leq d(C_u, C_v)$ for any subsequent target state C_v.*

Proof. Let the discrete transition cost between two physical magazine states be governed uniformly by the number of tool insertions required. If the origin state C_u operates under capacity ($|C_u| < b$), it contains exactly $b - |C_u|$ vacant slots.

We construct the padded maximal state C'_u by arbitrarily selecting a subset of unrequired tools T_{dummy} such that $|T_{dummy}| = b - |C_u|$, and applying the set union: $C'_u = C_u \cup T_{dummy}$. By definition, this guarantees that $|C'_u| = b$.

Consider a routing transition to an arbitrary subsequent maximal configuration $C_v \in \mathcal{C}$ where $|C_v| = b$. Because the original configuration is a strict subset of the padded configuration ($C_u \subset C'_u$), the intersection of the origin state with the target state satisfies the fundamental subset relation:

$$(C_u \cap C_v) \subseteq (C'_u \cap C_v). \tag{4.14}$$

Applying the cardinality operator strictly preserves this bounding inequality:

$$|C_u \cap C_v| \leq |C'_u \cap C_v|. \tag{4.15}$$

The mechanical transition cost into the target configuration C_v evaluates precisely to the capacity minus the shared intersection. Substituting inequality (4.15) into the metric function yields:

$$b - |C'_u \cap C_v| \leq b - |C_u \cap C_v| \implies d(C'_u, C_v) \leq d(C_u, C_v). \tag{4.16}$$

By structurally proving that $d(C'_u, C_v)$ acts as a lower bound to $d(C_u, C_v)$, we establish that injecting zero-cost dummy tools into vacant slots never degrades the global routing objective . Furthermore, by strictly forcing $|C_u| = b$ universally across the entire combinatorial domain, we lock all transition costs to the exact symmetric intersection metric required by the restricted GTSP mapping, successfully bypassing the Metric Barrier. □

4.3.3 Visualizing the Dummy Tool Padding Mapping

The mechanical engineering of the metric space via dummy padding is visually mapped in Figure 4.2. By artificially inflating the configuration vector to size b, the resulting integer transition weights can be safely passed to a classical GTSP solver without violating polyhedral symmetry.

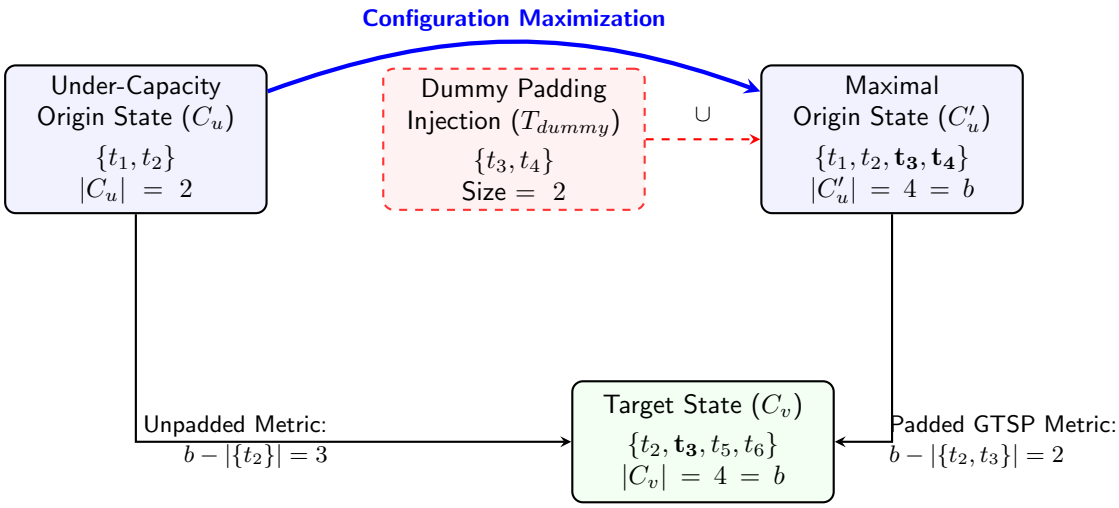

Figure 4.2: The Dummy Tool Padding mechanism. The under-capacity configuration C_u naturally incurs a high transition cost because it fails to exploit available magazine capacity $b = 4$. By padding the state with dummy tools $\{t_3, t_4\}$, the resulting maximal configuration C'_u serendipitously intersects with the future target state C_v. This padding explicitly lowers the symmetric integer transition cost from 3 to 2, locking the geometry into the required GTSP domain.

The mathematical consequences of the I-N transformation and Dummy Tool Padding complete the theoretical bridge. The U-SSP is now rigorously proven to be structurally homologous to a restricted Generalized Traveling Salesman Problem defined over maximal configuration clusters. To solve this exponentially large routing graph in industrial practice, we must now transition from graph mapping to exact polyhedral column generation frameworks.

Chapter 5

Discrete Batching vs. Continuous Routing (The JGP)

5.1 The Job Grouping Problem (JGP) as a Relaxation

To rigorously establish the polyhedral boundaries of the Job Sequencing and Tool Switching Problem (SSP), the continuous sequential routing space is frequently relaxed into a discrete batching domain. This structural relaxation is formally defined in the operations research literature as the Job Grouping Problem (JGP).

5.1.1 Formal JGP Formulation as a Set Covering Problem

The foundational premise of the JGP is to partition the job set J into mutually exhaustive subsets, referred to as batches or groups. The physical constraint of this domain dictates that all jobs assigned to a single batch must be processed together under one unified, static machine setup .

Mathematically, a specific subset of jobs $S \subseteq J$ is defined as a *feasible group* if and only if the absolute union of their required tool sets does not strictly exceed the machine's magazine capacity b. The feasibility condition is formally expressed as:

$$\left| \bigcup_{j \in S} T_j \right| \leq b. \tag{5.1}$$

Let $\mathcal{S}$ denote the exponential family of all possible feasible groups that can be constructed from the job set J. The objective of the JGP is to find a minimal cardinality partition of J using groups selected strictly from $\mathcal{S}$.

Crama and Oerlemans (1994) rigorously formulated this batching requirement as a classical Set Covering Problem .

We introduce a binary decision variable $y_S \in \{0, 1\}$, which equals 1 if the feasible group $S \in \mathcal{S}$ is selected to form a production batch, and 0 otherwise. The complete integer linear programming formulation for the JGP is defined as follows:

$$\text{Minimize} \quad Z_{JGP} = \sum_{S \in \mathcal{S}} y_S \tag{5.2}$$

$$\text{subject to} \quad \sum_{S \in \mathcal{S}: i \in S} y_S \geq 1 \qquad \forall i \in J \tag{5.3}$$

$$y_S \in \{0, 1\} \qquad \forall S \in \mathcal{S}. \tag{5.4}$$

The objective function (5.2) strictly minimizes the total number of active batches, which corresponds directly to minimizing the total number of complete machine setups required . Constraint (5.3) represents the standard set-covering inequality, mathematically mandating that every required job $i \in J$ is successfully assigned to at least one active batch.

Because the cardinality of the column space $|\mathcal{S}|$ grows exponentially with respect to the number of jobs and tools, explicit enumeration of all feasible groups is computationally intractable for industrial instances. Consequently, resolving the continuous linear relaxation of (5.2)–(5.4) strictly necessitates the deployment of advanced Dantzig-Wolfe decomposition and column generation frameworks .

5.2 Analytical Bounding and Divergence Analysis

While the Job Grouping Problem (JGP) enforces discrete operational halts, establishing a rigorous mathematical relationship between the optimal JGP batching cost (Z^*_{JGP}) and the optimal continuous SSP routing cost (Z^*_{SSP}) is critical for bounding the exact solution space. We can formally demonstrate that the continuous and discrete optimal values act as mutual, deterministic bounds for one another.

5.2.1 The α-Approximation Upper Bound

By analyzing the absolute worst-case transition penalties between mutually exclusive JGP batches, we can establish that the optimal discrete grouping solution yields an exact α-approximation for the continuous SSP sequence.

Theorem 5.1 (The α-Approximation Upper Bound)**.** *Let Z^*_{JGP} denote the minimum number of feasible batches required to partition the job set, and let Z^*_{SSP} denote the absolute minimum number of sequential tool switches*

required to process the jobs. The optimal continuous cost is strictly bounded from above by the inequality:

$$Z^*_{SSP} \leq b \cdot Z^*_{JGP}. \tag{5.5}$$

Proof. Let the optimal JGP solution partition the job set into exactly $k = Z^*_{JGP}$ disjoint, feasible groups: $G_1, G_2, \ldots, G_k$. By definition of a feasible group, the strict union of tool requirements for any group G_i satisfies $|T(G_i)| \leq b$.

Consider an arbitrary, continuous processing schedule for the U-SSP where the jobs are sequenced strictly according to these k batches: $G_1 \to G_2 \to \cdots \to G_k$. We evaluate the maximum possible switching penalty across this sequence. The absolute worst-case operational scenario occurs if the aggregated tool requirements of every two consecutive groups are perfectly disjoint, meaning $T(G_i) \cap T(G_{i+1}) = \emptyset$ for all i.

Under total disjointness, the CNC machine is mathematically forced to execute a complete magazine flush at every single batch boundary. Transitioning into group G_{i+1} requires evicting all previous tools and inserting exactly $|T(G_{i+1})|$ new tools. Assuming maximum capacity saturation (b) at every boundary to ensure a true upper bound, a sequence of k groups possesses exactly $k - 1$ inter-group transitions.

Including the initial loading of up to b tools for the first group G_1, the absolute maximum number of individual tool insertions across this sequence evaluates to:

$$\text{Total Switches} \leq b + b(k-1) = b \cdot k. \tag{5.6}$$

Because the continuous U-SSP model, optimized dynamically via the Keep Tool Needed Soonest (KTNS) retention policy, natively retains unneeded tools across batch boundaries whenever physical capacity permits, its true optimal continuous switching cost Z^*_{SSP} must be less than or equal to this rigid, worst-case batching cost. Substituting Z^*_{JGP} for k definitively proves the bounding inequality:

$$Z^*_{SSP} \leq b \cdot Z^*_{JGP}. \tag{5.7}$$

□

5.2.2 The Strict Continuous Lower Bound

Conversely, while the JGP partitions upper bound the SSP, the optimal continuous sequence strictly bounds the discrete batching problem from below. We can formally prove that the optimal number of discrete JGP batches can never exceed the total number of continuous tool switches plus one.

Theorem 5.2 (The Continuous Lower Bound)**.** *For any uniform Tool Switching Problem, the optimal discrete batching cost Z^*_{JGP} is bounded from above by the optimal continuous switching cost Z^*_{SSP} shifted by exactly one:*

$$Z^*_{JGP} \leq Z^*_{SSP} + 1. \tag{5.8}$$

Proof. Let $\sigma^* = (j_1, j_2, \ldots, j_n)$ be the globally optimal job sequence for the continuous U-SSP, incurring exactly Z^*_{SSP} total tool switches over the production horizon.

We can construct a valid, feasible JGP partition directly from this continuous sequence by tracking cumulative tool insertions. We initialize the first discrete batch G_1 by sequentially adding jobs $j_1, j_2, \ldots$ until the cumulative union of their required tools strictly exceeds the magazine capacity b. Let j_{i_1} be the exact job in the sequence where this capacity threshold is breached. We finalize the first batch as $G_1 = \{j_1, \ldots, j_{i_1-1}\}$.

Because the mathematical union of requirements for G_1 combined with the requirements for j_{i_1} exceeds b, the continuous SSP operation must have been mathematically forced to execute at least one physical tool eviction and insertion between the start of the sequence and the execution of job j_{i_1}. Therefore, the formation of this first batch boundary guarantees that at least 1 switch has been recorded in the continuous tally Z^*_{SSP}.

We repeat this greedy boundary formation for the remainder of the sequence, constructing groups $G_2, G_3, \ldots, G_k$. Each time a new group G_r is forced to close and G_{r+1} begins, the cumulative tool requirement has breached capacity, proving that at least one additional tool switch must have occurred in the continuous state space across that specific interval.

If this procedure constructs exactly k feasible groups, the continuous sequence must have incurred a minimum of $k - 1$ tool switches. Therefore, we establish the inequality:

$$Z^*_{SSP} \geq k - 1 \implies k \leq Z^*_{SSP} + 1. \tag{5.9}$$

Because k represents the cardinality of a heuristically constructed, valid JGP partition, the true optimal partition Z^*_{JGP} must be less than or equal to k. Thus, by transitivity, we conclude:

$$Z^*_{JGP} \leq k \leq Z^*_{SSP} + 1 \implies Z^*_{JGP} \leq Z^*_{SSP} + 1. \tag{5.10}$$

□

5.2.3 Analytical Conditions for Optimality Divergence

While the discrete Job Grouping Problem (JGP) provides mathematically robust upper and lower bounding mechanisms, the optimal configuration set generated by a JGP solver is not strictly guaranteed to serve as an optimal chassis for the continuous Uniform Tool Switching Problem (U-SSP). The polyhedral divergence between the discrete batching objective and the continuous routing objective is governed by specific topological conditions within the job-tool incidence matrix .

The optimal JGP configurations diverge from the optimal U-SSP routing path when the structural overlap of tool requirements is highly cyclical, triggering the following two analytical conditions:

1. **High Inter-Batch Tool Reuse (Bridge Conditions):** In a continuous routing environment, the Keep Tool Needed Soonest (KTNS) policy allows specific tools to "flow" seamlessly across multiple job executions without incurring setup penalties . If a tool is required in both the current theoretical batch and a distant future batch, its period of non-use constitutes a 0-block. If the physical magazine capacity permits, the optimal U-SSP sequence will prioritize retaining this "bridge" tool. Conversely, the JGP strictly enforces mutual exclusivity between batches, evaluating inter-batch transitions as complete mathematical flushes, thereby blindly severing the bridge and forcing an artificial double-penalty (eviction and subsequent re-insertion) for the same tool.

2. **Configuration Space Choking:** When the capacity threshold b is extremely tight relative to the average job requirement $|T_j|$, the magazine becomes "choked." To retain a bridge tool during a 0-block, the continuous SSP model must actively sacrifice a magazine slot that could otherwise be used to process the immediate job. JGP solvers, strictly optimizing for the absolute minimum number of batches, will greedily saturate all b slots with tools necessary to close the current batch as quickly as possible. By prioritizing batch compression over tool retention, the JGP fundamentally ignores the long-term sequential utility of tools .

Consequently, in environments characterized by highly overlapping, cyclical tool requirements, an exact optimal solution to the continuous SSP will deliberately select configurations that appear strictly suboptimal to a JGP solver because they prioritize overlapping tool retention over discrete batch minimization.

5.2.4 Visualizing the Routing vs. Batching Dichotomy

The suboptimality of the JGP when evaluated under a continuous routing environment is visually demonstrated in Figure 5.1, utilizing a cyclical requirement ring where the U-SSP dynamically bypasses the rigid capacity choke of the JGP boundaries.

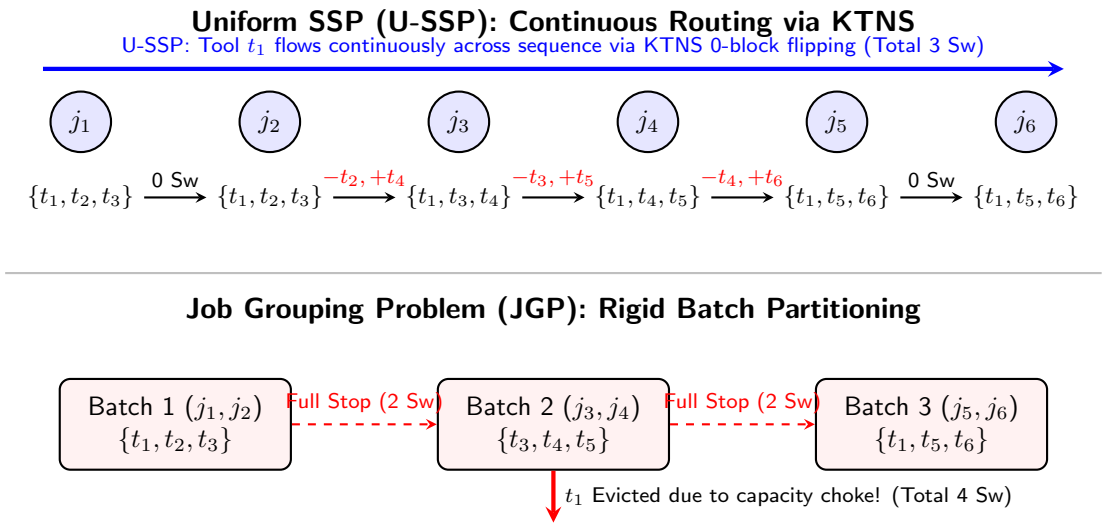

Figure 5.1: Graph-theoretic contrast between the Job Grouping Problem (JGP) and the Uniform Tool Switching Problem (U-SSP). The JGP acts strictly as a Set Covering problem, mapping jobs to rigid configuration nodes separated by complete machine stops, unnecessarily evicting bridge tool t_1. Conversely, the U-SSP models a continuous routing path, evaluating dynamic edge costs and allowing t_1 to flow seamlessly across the 0-block.

5.3 Heuristic Bridges: Breaking Batch Boundaries

5.3.1 Sequential Construction Heuristics for the JGP

Because the Set Covering formulation of the Job Grouping Problem (JGP) is itself NP-hard, explicit integer resolution is often computationally prohibitive for massive industrial instances . To quickly generate a valid upper bound for both the JGP and the underlying Tool Switching Problem (SSP), operations research practice relies heavily on sequential constructive heuristics. These heuristics build feasible job batches iteratively, operating under the rigid discrete batching paradigm where each group constitutes a complete machine setup.

The standard sequential construction framework operates via a two-step iterative logic. First, a "seed" job is selected to initiate a new batch S. Typically, the job requiring the highest absolute number of tools, $\max |T_i|$, is chosen to act as the heaviest anchor for the group. Second, a greedy selection rule is utilized to iteratively add unassigned jobs to S until the magazine capacity b prohibits any further additions (i.e., until the group is maximal) .

The efficacy of this construction heavily depends on the local selection rule used to evaluate tool overlap. For a current partial group S, let $T(S) =$

$\bigcup_{j \in S} T_j$ denote the aggregate tools already loaded in the magazine. For any unassigned candidate job $i \notin S$, let $t_i = |T_i|$ denote its absolute tool requirement, and let $b_i = |T_i \cap T(S)|$ denote the number of tools it shares with the current group.

The most prominent selection criteria established in the literature include:

- **MIMU (Maximal Intersection Minimal Union):** This rule selects the candidate job i that maximizes the intersection b_i. In the event of a tie, it breaks the symmetry by selecting the job that minimizes the number of additional tools required, evaluated strictly as $(t_i - b_i)$.
- **MI (Maximal Intersection):** A relaxed variant of MIMU that strictly maximizes b_i, resolving ties arbitrarily rather than explicitly minimizing the union penalty .
- **MU (Minimal Union):** This rule strictly minimizes the marginal tool additions $(t_i - b_i)$ required to accommodate the new job, prioritizing the preservation of remaining magazine capacity over maximizing immediate overlap .

Once the greedy procedure exhausts the unassigned job list, it yields a discrete partition $\mathcal{P} = \{G_1, G_2, \ldots, G_k\}$. Under the rigid batching assumption, the cost of this partition is precisely k machine setups.

5.3.2 Trajectory Algorithms and Batch Dissolution

While the JGP partition $\mathcal{P}$ provides a mathematically sound upper bound ($\text{OPT}_{SSP} \leq b \cdot |\mathcal{P}|$), it is strictly suboptimal for the continuous routing environment of the SSP because it artificially imposes rigid "batch walls." These walls blindly force the eviction of bridge tools that could otherwise be seamlessly carried over between consecutive groups .

To bridge the gap between discrete batching and continuous routing, modern solvers translate the discrete JGP sets into a continuous SSP upper bound utilizing local search trajectory algorithms. This process is initialized by creating a "warm-start" sequence $\sigma^{(0)}$. The jobs within each discrete batch $G_r \in \mathcal{P}$ are sequenced locally, and the batches themselves are concatenated to form a complete, continuous permutation $\sigma^{(0)} = (G_1 \to G_2 \to \cdots \to G_k)$.

When this concatenated sequence is evaluated dynamically using the continuous Keep Tool Needed Soonest (KTNS) policy, the KTNS algorithm natively identifies any serendipitous tool overlaps across the artificial batch boundaries, immediately recovering a tighter upper bound than the rigid JGP evaluation .

To actively dissolve the batch walls and further compress the sequence, trajectory methods systematically explore the permutational neighborhood $\mathcal{N}(\sigma)$.

1. **Local Search (2-Opt and 3-Opt):** By systematically swapping jobs across the boundaries of adjacent batches G_r and G_{r+1}, k-opt operators effectively shatter the mutual exclusivity of the original JGP sets . If swapping job $i \in G_r$ with job $j \in G_{r+1}$ creates a dense continuous 0-block that KTNS can exploit, the swap is accepted. This localized diffusion allows the configuration path to evolve smoothly, prioritizing overlapping tool retention over discrete batch isolation.

2. **Adaptive Large Neighborhood Search (ALNS):** To escape deep local optima caused by clustered batching, advanced frameworks deploy ALNS heuristics . ALNS employs aggressive "destroy" operators that systematically rip out large segments of the sequence (often entire batch transitions) and "repair" operators (such as Farthest Insertion or Regret-based insertion) to reweave the jobs into new, highly overlapping continuous sequences.

By leveraging the JGP solely as an intelligent spatial initialization and relying on KTNS-evaluated trajectory methods to organically dissolve the boundaries, the solver effectively bridges the discrete and continuous topological domains, rapidly yielding near-optimal continuous routing paths.

Chapter 6

Non-Uniform Setup Costs and Tool-Dependent Dynamics

6.1 The Privault-Finke Variant (TD-SSP)

6.1.1 Tool-Dependent Dynamics and the Breakdown of KTNS

The foundational mathematical analysis of the Tool Switching Problem strictly assumes a uniform architecture, where the mechanical effort and temporal penalty required to insert or evict any individual tool are evaluated identically However, industrial reality frequently necessitates the evaluation of non-uniform setup times, a paradigm formally defined as the Tool-Dependent SSP (TD-SSP) In this variant, extensively studied by Privault and Finke, the transition costs are sequence-dependent, meaning the precise cost d_{hk} depends explicitly on the specific tool h being removed from the magazine and the new tool k being inserted

Under this non-uniform paradigm, the fundamental greedy choice property of the Keep Tool Needed Soonest (KTNS) policy collapses completely In the standard uniform case, the deterministic optimal choice is to evict the tool whose next required use is farthest in the future. However, when setup costs are highly asymmetric, an irreconcilable conflict emerges between temporal proximity and mechanical penalty. The decision to retain a tool that is needed far into the future may suddenly become mathematically optimal if its specific mechanical replacement penalty is astronomically high relative to the other tools currently occupying the slots. Because the KTNS procedure relies on uniform cost assumptions, it is fundamentally blind to these asymmetries and is no longer a valid optimal strategy for the tooling subproblem

6.1.2 Minimum-Cost Flow and Total Unimodularity

Despite the failure of the greedy KTNS heuristic, the underlying tooling subproblem for a fixed sequence remains solvable in exact polynomial time Privault and Finke established that determining the optimal non-uniform tool loading policy can be mathematically formulated as a max-flow-min-cost model This approach maps the continuous retention of tools to an acyclic network flow, where capacity constraints reflect the magazine limit and edge weights incorporate the asymmetric changeover costs d_{hk}

The mathematical elegance of this subproblem lies in its polyhedral geometry. The constraint matrix governing the linear programming formulation of this network behaves structurally as an interval matrix An incidence matrix is defined as an interval matrix if it satisfies the Consecutive Ones Property (COP), meaning its rows can be permuted such that all non-zero entries in every column occur contiguously.

By the fundamental theorems of combinatorial optimization, interval matrices are mathematically guaranteed to be Totally Unimodular (TU) A constraint matrix $\mathbf{A}$ is TU if every square submatrix possesses a determinant evaluating strictly to 0, +1, or −1. Because the interval matrix of the non-uniform tooling subproblem is totally unimodular, the continuous linear programming (LP) relaxation natively yields exact integer optimal solutions

$$\mathbf{A}x = \mathbf{b}, \quad x \geq 0 \implies x^* \in \mathbb{Z}^n \tag{6.1}$$

Consequently, the non-uniform tooling subproblem completely bypasses the need for branch-and-bound integer resolution. It can be solved to absolute integer optimality in polynomial time by simply evaluating the continuous linear programming relaxation of the formulation, preserving the Tang-Denardo decomposition paradigm even under highly asymmetric setup penalties

6.2 Lexicographical Minimization to the JGP

6.2.1 Definition of Heavy Machine-Stop Penalties

In industrial flexible manufacturing environments, the time or financial penalty associated with completely halting a CNC machine to perform a batch tool change often vastly exceeds the cost of executing an individual, sequential tool switch . Let C_{stop} denote the massive fixed penalty incurred whenever the continuous machine sequence is structurally broken to initiate a full magazine flush, and let c_{switch} represent the maximum marginal cost of a single tool exchange under the Tool-Dependent SSP (TD-SSP) framework.

When the setup matrix exhibits extreme operational asymmetry such that the fixed stop penalty dominates the marginal switching costs, the continuous TD-SSP routing objective undergoes a radical topological shift .

We formalize this heavy penalty regime by establishing the following strict bounding inequality:

$$C_{stop} \gg c_{switch} \cdot b \cdot n \tag{6.2}$$

where b is the magazine capacity and $n = |J|$ is the total number of jobs. Under these specific conditions, the optimization problem naturally bifurcates into a strictly ordered hierarchy of primary and secondary objectives .

6.2.2 Theorem and Proof: Lexicographical Recovery of the JGP

We can now formally prove that under the heavy penalty condition, the continuous sequence optimization of the TD-SSP structurally collapses back into the exact discrete partitioning objective of the Job Grouping Problem (JGP).

Theorem 6.1 (Lexicographical JGP Recovery)**.** *For any TD-SSP instance where the machine halt penalty satisfies $C_{stop} > n \cdot b \cdot \max(c_{switch})$, the optimal operational routing policy is lexicographically identical to the optimal solution of the corresponding discrete Job Grouping Problem (Z^*_{JGP}).*

Proof. Let a valid operational sequence for the TD-SSP be partitioned into exactly k contiguous, non-overlapping operational batches separated by full machine stops. The total objective cost Z_{TD} evaluated over this entire sequence is:

$$Z_{TD} = k \cdot C_{stop} + \sum_{r=1}^{k} S_r \tag{6.3}$$

where S_r represents the aggregate marginal tool switching costs incurred strictly within the continuous boundaries of batch r.

Because any single job $j \in J$ requires at most b tools to be loaded, the absolute theoretical maximum number of individual tool switches across the entire continuous production sequence of n jobs is strictly bounded by $n \cdot b$. Consequently, the total internal switching cost across all batches is deterministically bounded:

$$\sum_{r=1}^{k} S_r \leq n \cdot b \cdot \max(c_{switch}). \tag{6.4}$$

By our foundational premise defining the heavy penalty regime, we have established that $C_{stop} > n \cdot b \cdot \max(c_{switch})$. Substituting the theoretical upper bound from inequality (6.4) directly yields:

$$\sum_{r=1}^{k} S_r < C_{stop}. \tag{6.5}$$

Now, consider two competing valid sequences: sequence σ_A operating with exactly k machine stops, and sequence σ_B operating with $k+1$ machine stops. The total cost of sequence σ_B is evaluated as:

$$Z_{TD}(\sigma_B) = (k+1)C_{stop} + S_B = k \cdot C_{stop} + C_{stop} + S_B. \tag{6.6}$$

Because the single scalar value C_{stop} strictly exceeds the absolute maximum possible cumulative marginal switching cost of any sequence (S_A), we possess the strict inequality $C_{stop} > S_A$. Adding the shared base cost $k \cdot C_{stop}$ to both sides yields:

$$k \cdot C_{stop} + C_{stop} + S_B > k \cdot C_{stop} + S_A. \tag{6.7}$$

Therefore, the inequality $Z_{TD}(\sigma_B) > Z_{TD}(\sigma_A)$ universally holds, regardless of how efficiently the internal sequential switches S_B are optimized .

This rigorous bounding proves that the objective function is strictly lexicographical. The primary objective must absolutely minimize the discrete scalar k (the total number of full machine stops) before the secondary objective (minimizing the internal marginal switches S) can even mathematically influence the global optimum . Because minimizing the absolute number of feasible, disjoint batches k is the exact, unrelaxed mathematical definition of the Job Grouping Problem objective (Z^*_{JGP}), the TD-SSP under heavy penalty structurally subsumes and identically matches the JGP partition. □

Chapter 7

Exact Resolution via Branch-and-Price

7.1 The Restricted Master Problem (RMP) with GSECs

Because the fundamental configuration space $\mathcal{C}$ constructed for the Uniform Tool Switching Problem (U-SSP) is bounded by the binomial coefficient $\binom{m}{b}$, the full transition graph scales exponentially with the magazine capacity and total tool set Consequently, evaluating the complete combinatorial state space directly via standard continuous linear programming is computationally prohibitive for industrial instances.

To achieve exact resolution, the non-compact mathematical formulation is fundamentally restructured through Dantzig-Wolfe decomposition into a Branch-and-Price (B&P) architecture. This approach partitions the computational burden: a Restricted Master Problem (RMP) evaluates a limited, dynamically generated subset of configuration columns, while a Pricing Oracle systematically explores the state space to discover strictly improving configurations

7.1.1 Formulation of the Linear Relaxation of the RMP

Let $\mathcal{C}_{subset} \subset \mathcal{C}$ represent the active, limited subset of maximal tool configurations currently known to the solver. The RMP operates strictly as a continuous linear relaxation over this restricted domain. We relax the integrality constraints on the binary variables, defining continuous decision variables $y(v) \in [1]$ for the activation of a configuration $v \in \mathcal{C}_{subset}$, and $x(u, v) \in [1]$ for the directed routing transition between configuration u and configuration v.

The RMP seeks a minimum-cost fractional routing path through this restricted topology. The mathematical model is rigorously formulated as

follows:

$$\text{Minimize} \quad Z_{RMP} = \sum_{u \in \mathcal{C}_{subset}} \sum_{v \in \mathcal{C}_{subset}} d(u,v)x(u,v) \tag{7.1}$$

$$\text{subject to} \quad \sum_{v \in H_j \cap \mathcal{C}_{subset}} y(v) \geq 1 \qquad \forall j \in J \tag{7.2}$$

$$\sum_{u \in \mathcal{C}_{subset}} x(u,v) = y(v) \qquad \forall v \in \mathcal{C}_{subset} \tag{7.3}$$

$$\sum_{w \in \mathcal{C}_{subset}} x(v,w) = y(v) \qquad \forall v \in \mathcal{C}_{subset} \tag{7.4}$$

$$\sum_{u \in H(S)} \sum_{v \in H(S)} x(u,v) \leq \sum_{v \in H(S)} y(v) - 1 \quad \forall S \subset J,\ 2 \leq |S| \leq |J| - 1 \tag{7.5}$$

$$y(v) \geq 0, \quad x(u,v) \geq 0 \qquad \forall u, v \in \mathcal{C}_{subset} \tag{7.6}$$

The objective function (7.1) minimizes the total cumulative transition costs evaluated via the symmetric intersection metric $d(u,v) = b - |C_u \cap C_v|$. Constraint (7.2) constitutes the Set Covering boundary, strictly ensuring that every job $j \in J$ is mathematically satisfied by at least one active configuration residing within its feasibility cluster H_j.

Constraints (7.3) and (7.4) maintain strict flow conservation; if a fractional weight is assigned to a configuration node, an identical sum of continuous flow must enter and exit that vertex. Finally, inequality (7.5) dynamically applies the Generalized Subtour Elimination Constraints (GSECs) over the union of job clusters $H(S) = \bigcup_{j \in S} H_j$ restricted to $\mathcal{C}_{subset}$. This strictly prohibits the formation of disconnected fractional cycles isolated from the main Hamiltonian path.

7.1.2 Extraction of the Comprehensive Dual Variable Vector

The continuous optimization of the RMP provides critical polyhedral pricing information to the subproblem via its shadow prices. Because the structural matrix includes coverage, conservation, and subtour constraints, the resulting dual space is multidimensional. We formally define the exact dual vector $\Theta = \langle \pi, \alpha, \beta, \mu \rangle$ extracted from the LP relaxation:

1. **Coverage Duals ($\pi_j \geq 0$):** Associated with constraint (7.2), π_j represents the marginal economic reward (the "bounty") for successfully

covering job $j \in J$. Because the primal constraint is a "greater-than-or-equal-to" inequality in a minimization problem, the corresponding dual variables are strictly non-negative.

2. **Flow Conservation Duals ($\alpha_v, \beta_v \in \mathbb{R}$):** Associated with the inflow (7.3) and outflow (7.4) equations respectively. Because the primal boundaries are strict equalities, these dual variables are unrestricted in sign. They represent the topological balancing forces required to maintain continuity through configuration v.

3. **Subtour Penalty Duals ($\mu_S \leq 0$):** Associated with the GSEC inequalities (7.5). Because these are "less-than-or-equal-to" constraints governing the internal edge density of subset S, their corresponding dual variables must be mathematically non-positive. They act as severe economic penalties applied to any dynamically generated transitions that attempt to localize flow exclusively within the subset $H(S)$.

7.1.3 Ryan-Foster Branching Logic over Configuration Pairs

Because the RMP evaluates a continuous relaxation, an optimal fractional solution Z^*_{RMP} must eventually be resolved into an integer sequence. Standard binary branching on individual variables (e.g., $x(u, v) = 1 \vee 0$) heavily disrupts the structural geometry of the pricing subproblem

Instead, the framework utilizes the Ryan-Foster branching logic, uniquely adapted for configuration subsets. If the LP returns a fractional allocation, the algorithm identifies two distinct jobs $i, k \in J$ that are consecutively processed by a shared fractional sequence of configurations. The Ryan-Foster rule legally partitions the search space by imposing the following dichotomy:

1. **Branch 1 (Together):** The configuration sequences satisfying jobs i and k must be executed adjacently. The RMP merges their clusters, and the pricing subproblem is restricted to generating columns that satisfy both requirements sequentially.

2. **Branch 2 (Separated):** Jobs i and k are forbidden from being processed adjacently. Any transition edge $x(u, v)$ connecting H_i directly to H_k is purged, and the oracle is mathematically restricted from generating paths containing this transition.

This highly balanced branching strategy eliminates fractional cycling without destroying the polynomial structure of the underlying pricing oracle.

7.2 The Node-Pricing Oracle as a Hypergraph-SUKP

7.2.1 The H-SUKP Architecture and Edge Reduced Cost

Within the Branch-and-Price (B&P) framework, the dynamic generation of columns operates over a highly structured hypergraph topology $\mathcal{H} = (T, J)$. The physical tools act as the fundamental nodes $t \in T$, and the job requirements act as hyperedges $j \in J$ connecting their requisite tools The Pricing Oracle must construct a new maximal configuration $v \subset T$ (where $|v| = b$) that maximizes the collection of coverage duals π_j supplied by the Restricted Master Problem (RMP). Because a hyperedge j is only successfully "covered" if all of its constituent tool nodes are simultaneously included within the knapsack capacity b, the node-generation oracle maps identically to the strongly NP-hard Hypergraph Set Union Knapsack Problem (H-SUKP)

Once an optimal configuration vertex v is generated by the H-SUKP oracle, the continuous linear relaxation must mathematically evaluate the economic viability of routing continuous flow into and out of v. By extracting the dual vector $\Theta = \langle \pi, \alpha, \beta, \mu \rangle$ from the RMP, we can rigorously derive the exact reduced cost $\bar{c}_{uv}$ for any dynamically generated transition edge $x(u, v)$ connecting an existing configuration $u \in \mathcal{C}_{subset}$ to the newly generated configuration v.

Based on the fundamental principles of linear programming duality, the reduced cost of the fractional flow variable evaluates to the original primal transition metric minus the algebraic sum of the dual variables associated with the specific constraints in which $x(u, v)$ actively participates. Specifically, the variable $x(u, v)$ explicitly appears in the outflow conservation constraint for u (with dual α_u), the inflow conservation constraint for v (with dual β_v), and critically, the Generalized Subtour Elimination Constraints (GSECs) for any topological subset $S \subset J$ where both configurations uniquely reside within the subset's overlapping feasibility cluster $H(S)$.

The exact continuous edge reduced cost is thus formally defined as:

$$\bar{c}_{uv} = d(u, v) - \alpha_u - \beta_v - \sum_{S \subset J \,:\, u,v \in H(S)} \mu_S \tag{7.7}$$

Because the master problem strictly minimizes the global routing objective, the oracle seeks to identify and append fractional transitions where $\bar{c}_{uv} < 0$. However, the continuous geometry of the non-compact space forces this generation process to simultaneously negotiate the subtour penalties governed by μ_S.

7.2.2 Mathematical Interaction: GSEC Duals as Absolute Penalties

The integration of Generalized Subtour Elimination Constraints directly into the continuous RMP formulation introduces a profound polyhedral regulatory mechanism over the dynamic column generation process. We can formally prove that the GSEC dual variables intrinsically force the H-SUKP Pricing Oracle to natively avoid generating densely localized, fractional subtours without requiring external branching interventions.

Theorem 7.1 (GSEC Dual Penalization)**.** *Let μ_S be the optimal dual variable associated with the Generalized Subtour Elimination Constraint for a specific job subset $S \subset J$. For any dynamically generated transition edge $x(u, v)$ strictly internal to the overlapping feasibility cluster $H(S)$ (meaning $u, v \in H(S)$), the dual variable μ_S acts as a mathematically absolute, non-negative penalty added directly to the edge's reduced cost, effectively suppressing its generation.*

Proof. By the fundamental polyhedral definitions of the primal RMP, the GSEC for any valid subset S operates strictly as a "less-than-or-equal-to" inequality, explicitly engineered to restrict the internal fractional flow density:

$$\sum_{u \in H(S)} \sum_{v \in H(S)} x(u, v) \leq \sum_{v \in H(S)} y(v) - 1 \tag{7.8}$$

Because the primal continuous formulation seeks to minimize the global objective function, classical linear programming duality strictly mandates that the optimal dual variable associated with any $\leq$ inequality constraint must be non-positive Therefore, we establish the absolute theoretical guarantee that:

$$\mu_S \leq 0 \quad \forall S \subset J. \tag{7.9}$$

Consider the mathematical evaluation of a proposed internal transition edge $x(u, v)$ where both the origin configuration u and the destination configuration v are capable of processing jobs within S, thereby satisfying the cluster membership $u, v \in H(S)$. According to the exact reduced cost derivation established previously:

$$\bar{c}_{uv} = d(u, v) - \alpha_u - \beta_v - \sum_{\Omega \subset J \,:\, u,v \in H(\Omega)} \mu_\Omega \tag{7.10}$$

Because the specific job subset S is deterministically included in the family of sets Ω containing both vertices, the exact term $-\mu_S$ is explicitly injected into the algebraic evaluation of $\bar{c}_{uv}$. Given that the bounding dual variable satisfies $\mu_S \leq 0$, it mathematically follows that its negation is strictly non-negative:

$$-\mu_S \geq 0. \tag{7.11}$$

Consequently, the algebraic term $-\mu_S$ operates strictly as a non-negative penalty vector. Rather than subsidizing the cost of the transition, the GSEC dual mathematically inflates the perceived cost of the internal edge $x(u, v)$. The denser the internal fractional flow within the cluster $H(S)$ during the RMP evaluation, the more severely negative the dual variable μ_S becomes, resulting in a massively positive penalty $-\mu_S$ applied to any subsequent internal edge generation proposed by the oracle.

This rigorous structural interaction definitively proves that the GSECs do not merely prune existing fractional cycles within the RMP; they project a continuous polyhedral "force field" directly into the H-SUKP Pricing Oracle. The oracle is mathematically disincentivized from proposing transition edges that connect heavily overlapping, localized configurations, natively forcing the dynamic generation sequence to explore external transitions and strictly preserve the Hamiltonian topology of the global state space. □

7.3 Advanced Tightening Inequalities

To effectively accelerate the Branch-and-Price (B&P) convergence on massive industrial instances, the continuous linear relaxation of the Restricted Master Problem (RMP) must be fortified with advanced polyhedral cutting planes. We formalize two novel tightening mechanisms that directly exploit the underlying set-theoretic geometry of the Tool Switching Problem: the Tool Overlap Density (TOD) metric for Generalized Subtour Elimination Constraints (GSECs), and static Symmetry-Breaking constraints based on requirement cardinality.

7.3.1 Tool Overlap Density (TOD) in GSECs

The standard GSEC formulation treats all configurations as independent abstract vertices, effectively ignoring the varying degrees of mechanical similarity between the underlying tool sets. By mathematically quantifying this similarity, we can enforce strict, sequence-independent lower bounds on the transition costs within any localized subset of jobs.

We define the *Tool Overlap Density* (TOD) between any two distinct jobs $i, j \in J$ as the normalized cardinality of their mathematical intersection relative to the magazine capacity:

$$\rho_{ij} = \frac{|T_i \cap T_j|}{b} \tag{7.12}$$

Theorem 7.2 (TOD Transition Lower Bound)**.** *For any valid continuous routing sequence serving a subset of jobs $S \subseteq J$, the absolute minimum transition cost incurred between any configuration satisfying job $i \in S$ and any configuration satisfying job $j \in S$ is strictly bounded from below by $b(1 - \rho_{ij})$.*

Proof. Let C_u be an arbitrary valid configuration satisfying job i (meaning $T_i \subseteq C_u$) and let C_v be an arbitrary valid configuration satisfying job j (meaning $T_j \subseteq C_v$). By the fundamental axioms of set theory, the intersection of the supersets must be greater than or equal to the intersection of their subsets:

$$|C_u \cap C_v| \geq |T_i \cap T_j|. \tag{7.13}$$

The exact mechanical transition cost between the two maximal configurations evaluates to $d(u, v) = b - |C_u \cap C_v|$. Substituting the bounding intersection inequality yields:

$$d(u, v) \leq b - |T_i \cap T_j|. \tag{7.14}$$

Correction: Because we are subtracting a potentially smaller quantity, the inequality direction naturally reverses. The transition cost is bounded from below:

$$d(u, v) \geq b - |T_i \cap T_j|. \tag{7.15}$$

By factoring out the magazine capacity b and substituting the formal definition of the TOD metric ρ_{ij}, we establish the definitive lower bound:

$$d(u, v) \geq b \left(1 - \frac{|T_i \cap T_j|}{b}\right) = b(1 - \rho_{ij}). \tag{7.16}$$

□

This theorem allows us to construct a fully connected density graph over any job subset $S \subset J$, where the static edge weights are pre-computed as $b(1-\rho_{ij})$. Any continuous fractional flow satisfying S must incur a transition cost at least as large as the Minimum Spanning Tree (MST) of this density graph. Injecting this clique-based MST scalar directly into the objective function of the RMP's fractional flow matrices constitutes a mathematically valid, polyhedral tightening cut, elevating the lower bound significantly in highly heterogeneous manufacturing environments.

7.3.2 Symmetry-Breaking Constraints via Requirement Cardinality

Beyond fractional cycling, B&P frameworks suffer from massive search tree degradation due to permutational symmetry. In the Uniform SSP, the mechanical transition metric is perfectly symmetric: $d(u, v) = d(v, u)$. Consequently, any optimal continuous job sequence $\sigma^* = (\sigma_1, \sigma_2, \ldots, \sigma_n)$ deterministically possesses a dual optimal sequence traversed in strict reverse, $\sigma'_{reversed} = (\sigma_n, \ldots, \sigma_2, \sigma_1)$, which guarantees the exact same objective switching cost.

To halve the exponential dimension of the branch-and-bound tree, we can impose a static symmetry-breaking constraint that anchors the routing topology without risking the elimination of the global optimum.

Theorem 7.3 (Maximum Cardinality Anchoring)**.** *Let $j^* \in J$ be the specific job where the absolute tool requirement cardinality is maximized, such that $|T_{j^*}| \geq |T_i|$ for all $i \in J$. Constraining the job sequence such that j^* is strictly processed in the first half of the sequence (i.e., at an index $k \leq \lfloor n/2 \rfloor$) preserves at least one exact global optimum.*

Proof. Assume the continuous configuration space is fully symmetric. If a B&P solver identifies a globally optimal routing path where the maximum-cardinality job j^* is processed in the second half of the sequence (at index $k > \lfloor n/2 \rfloor$), the inherent symmetry of the U-SSP transition metric guarantees the existence of a reversed, parallel path evaluated at identical cost. In this reversed path, the processing position of j^* maps deterministically to $n - k + 1$, which mathematically evaluates to $\leq \lfloor n/2 \rfloor$.

Because the maximum-cardinality job severely restricts the available localized feasibility clusters H_{j^*} due to its massive tool saturation, anchoring it early in the sequence forces the Pricing Oracle to explore the most constrained, highly bounded regions of the polyhedral space immediately. □

By injecting this static symmetry-breaking rule into the initial formulation, the solver immediately prunes the redundant half of the symmetric search tree. This eliminates the evaluation of reversed fractional combinations and exponentially accelerates the exact resolution of the SSP.

Chapter 8

Neural-Augmented Column Generation Architectures

8.1 Machine Learning Taxonomy and Neural Oracles

8.1.1 Taxonomy of ML Interventions in Combinatorial Optimization

The integration of Machine Learning (ML) into Operations Research (OR) has fundamentally altered the resolution trajectories for NP-hard combinatorial optimization problems. To rigorously situate our proposed algorithmic enhancements for the Tool Switching Problem (SSP), we must classify the intervention within the standard OR-ML taxonomy [1, 2] .

Karimi-Mamaghan et al. (2022) structurally categorize ML interventions into three primary domains: end-to-end learning, learning to configure algorithms, and learning alongside optimization algorithms [3, 4] . End-to-end learning models seek to output the exact solution directly from the problem parameters, typically employing sequence-to-sequence architectures or graph neural networks trained via reinforcement learning

Learning to configure algorithms encompasses offline algorithm selection and dynamic parameter control, utilizing historical data to tune search hyper-parameters or select appropriate metaheuristics

However, the most theoretically robust paradigm for exact resolution is learning alongside optimization algorithms. In this framework, ML models are embedded directly within classical exact methods, such as Branch-and-Bound or Dantzig-Wolfe decomposition

8.1.2 Neural Sequence Oracles for H-SUKP Feasibility

Within the non-compact Branch-and-Price architecture for the SSP, the primary computational bottleneck is the exact resolution of the Pricing Sub-

problem. Because the dynamic generation of columns mandates solving the Hypergraph Set Union Knapsack Problem (H-SUKP) at every master iteration, exact MILP pricing solvers must repeatedly explore massive, exponential Branch-and-Bound trees to evaluate capacity-feasible tool intersections.

To bypass this exponential search space, we conceptualize the deployment of Neural Sequence Oracles under the paradigm of Neural Column Generation (NCG) [15, 16] . By framing the pricing subproblem as a sequential decision-making process, we can deploy Attention-based sequence models or Graph Neural Networks (GNNs) to rapidly evaluate the bipartite job-tool state matrix [17, 18] .

In this NCG pipeline, the neural sequence oracle receives the current structural state of the Restricted Master Problem (RMP), prominently including the continuous dual variable vector π. Instead of exhaustively enumerating tool subsets, the attention mechanism dynamically computes the statistical probability that a specific subset of tools will yield a configuration with a strictly negative reduced cost [19, 20] . The model leverages its learned weights to prioritize tools that cover high-bounty jobs (large π_j) while intrinsically recognizing the overlapping requirements that minimize knapsack capacity consumption [21, 22] .

Crucially, this neural oracle acts strictly as an informed heuristic filter [23, 24] . It bypasses the exact H-SUKP feasibility checking by predicting and constructing highly promising columns in polynomial time. These heuristically generated columns are subsequently injected into the continuous RMP. The mathematical exactness of the global Branch-and-Price algorithm is preserved via a strict hybrid fallback protocol: only when the Neural Sequence Oracle fails to identify a configuration with a negative reduced cost does the system instantiate the exact MILP solver to either find a deeply hidden column or rigorously prove that the continuous LP relaxation has reached global optimality [25, 26] .

8.2 Hypergraph Neural Networks for H-SUKP Pruning

8.2.1 Bipartite GNN Embeddings

To mathematically construct the Neural Sequence Oracle, the Restricted Master Problem (RMP) must be mapped into a topological state space that a neural architecture can process. Because the RMP natively defines a covering relationship between available configurations and job requirements, the entire solver state at iteration k can be rigorously modeled as a dynamic bipartite graph $\mathcal{B}_k = (\mathcal{V}_C, \mathcal{V}_J, \mathcal{E})$.

In this bipartite representation, the set $\mathcal{V}_C$ contains nodes representing the active configurations (columns) currently residing in the RMP basis, and

the set $\mathcal{V}_J$ contains nodes representing the job covering constraints (rows). A structural edge $e \in \mathcal{E}$ connects a configuration $c \in \mathcal{V}_C$ to a job $j \in \mathcal{V}_J$ if and only if the configuration satisfies that job's tool requirements ($T_j \subseteq c$) .

Crucially, we augment this bipartite graph by injecting a third set of nodes $\mathcal{V}_T$, representing the fundamental tools available in the magazine. We instantiate the continuous dual variables π_j, extracted directly from the optimal LP relaxation of the current RMP, as the primary scalar node features for $j \in \mathcal{V}_J$. Furthermore, we project the Generalized Subtour Elimination Constraint (GSEC) dual penalties $-\mu_S$ onto the hyperedges connecting the jobs within subset S.

We deploy a multi-layer Graph Neural Network (GNN) to execute localized message passing across this enriched topology. Let $h_v^{(l)}$ denote the high-dimensional latent embedding of node v at the l-th convolutional layer. The embedding update for a tool node $t \in \mathcal{V}_T$ aggregates the synergistic dual prizes of all jobs that require it, rigorously formulated as:

$$h_t^{(l)} = \mathrm{MLP}_{\theta_T}\left((1+\epsilon)\cdot h_t^{(l-1)} + \sum_{j\in\mathcal{N}(t)} \mathrm{MLP}_{\theta_J}\left(h_j^{(l-1)}\right)\right) \tag{8.1}$$

where $\mathcal{N}(t)$ defines the neighborhood of jobs requiring tool t, and ϵ acts as a structural residual weight .

After L layers of message passing, the network achieves a global receptive field, meaning the final embedding $h_t^{(L)}$ of each individual tool mathematically encapsulates the global continuous geometry of the RMP dual space. A feed-forward readout mechanism subsequently maps these final embeddings into a probability distribution $P(t \in C^*)$, predicting the exact likelihood that tool t belongs to the optimal negative reduced-cost configuration C^* . By thresholding these probabilities, the solver systematically discards tools with vanishingly low likelihoods, violently pruning the exponential H-SUKP search space before the exact MILP pricing solver is even invoked.

8.2.2 Actor-Critic RL for Diverse Column Selection

While the GNN effectively identifies tools to construct high-quality configurations, generating and injecting only the single column with the most negative reduced cost severely cripples convergence speed. In highly constrained combinatorial topologies like the Tool Switching Problem, iterative single-column generation induces severe primal degeneracy and wild oscillations in the dual vector space (the classical "tailing-off" effect).

To circumvent this, we must transition from greedy single-column generation to multiple-column injection. However, selecting an arbitrary batch of negative reduced cost columns often results in highly correlated, redundant configurations that crowd the RMP without improving the objective bound. Therefore, the column selection phase is fundamentally a sequential

decision-making process formulated as a Markov Decision Process (MDP) and solved via Deep Reinforcement Learning (RL) .

Let the action a_τ at iteration τ be the discrete selection of a subset of k diverse configurations from the pool of heuristically generated candidates. We train an Actor-Critic architecture utilizing Proximal Policy Optimization (PPO) to maximize a highly specialized reward function r_τ. The reward must explicitly penalize algorithmic stalling while simultaneously rewarding both objective minimization and structural column diversity .

The reward function evaluates the action a_τ exactly as:

$$r_\tau = -1 + \alpha \left(\frac{Z_{RMP}^{(\tau-1)} - Z_{RMP}^{(\tau)}}{Z_{RMP}^{(0)}} \right) + \beta \sum_{u,v \in a_\tau} \left(1 - \frac{\langle u, v \rangle}{\|u\| \|v\|} \right) \tag{8.2}$$

Here, the -1 term is a strict unit penalty for executing an additional column generation iteration. The parameter α scales the normalized fractional improvement in the RMP objective value Z_{RMP}. Critically, the parameter β scales a diversity maximization term calculated via the sum of the cosine distances between the binary configuration vectors u and v selected in the action batch a_τ .

During the execution of the Branch-and-Price loop, the Actor network evaluates the GNN embeddings of the candidate columns and outputs a policy distribution $\pi_\theta(a_\tau | s_\tau)$. By sampling from this distribution, the RL agent intelligently selects a batch of configurations that are highly orthogonal to one another but collectively exhibit strong negative reduced costs. This learned topological diversification mathematically stabilizes the multi-dimensional dual space $\Theta = \langle \pi, \alpha, \beta, \mu \rangle$, bypassing degeneracy and exponentially accelerating the convergence of the continuous LP relaxation toward global optimality.

8.3 Regression Bounding and Experimental Roadmap

8.3.1 Regression Bounding for the Exact MILP Oracle

While Neural Sequence Oracles and GNN-based topological pruning exponentially accelerate the discovery of negative reduced-cost columns, the absolute mathematical integrity of the Branch-and-Price (B&P) framework fundamentally relies on an exact fallback mechanism. If the heuristic neural agents fail to generate a column with a strictly negative reduced cost, the algorithm must invoke an exact Mixed-Integer Linear Programming (MILP) solver to evaluate the remaining Hypergraph Set Union Knapsack Problem (H-SUKP) space. This ensures the solver either discovers a deeply hidden optimal column or rigorously proves that the continuous LP relaxation has reached global optimality

However, solving the exact MILP for the H-SUKP from scratch at every fallback iteration is computationally devastating. To mitigate this, we integrate an online machine learning regression model designed to predict a tight objective bound for the exact pricing subproblem, heavily accelerating its internal branch-and-bound pruning

Because the RMP modifies the dual variable vector π continuously, the solutions to consecutive pricing iterations are highly correlated. Let $D = \{(\phi^{(i)}, f(x^{(i)}))\}_{i=1}^{n}$ represent a historical dataset collected during the B&P execution, where $\phi^{(i)}$ is a feature vector extracted directly from the RMP duals at iteration i, and $f(x^{(i)})$ is the exact minimum reduced cost subsequently found by the MILP

We train a dynamic, online regression model to map the current dual features ϕ to a predictive hypothesis $\hat{f}(\phi) = w^T\phi$, which estimates the minimum expected reduced cost for the current iteration

8.3.2 The Four-Phase Experimental Roadmap

The theoretical architectures and polyhedral bounding mechanisms established throughout this monograph pave the way for a highly structured, scalable implementation strategy. To systematically validate the integration of machine learning within the exact resolution of the Tool Switching Problem, we propose a formal 4-phase experimental roadmap

1. **Phase 1: Pure Operations Research (OR) Baseline.** The framework is initialized using classical Dantzig-Wolfe decomposition. The Restricted Master Problem (RMP) is formulated as a continuous Set Covering model, and the pricing oracle is resolved entirely via a standard exact MILP solver for the H-SUKP. High-quality initial columns are supplied exclusively via classical Multi-Start greedy heuristics and Local Search trajectory methods.

2. **Phase 2: Polyhedral Tightening and Exact ML Bounding.** The mathematical geometry of the solver is fortified. Tool Overlap Density (TOD) lower bounds are injected into the Generalized Subtour Elimination Constraints (GSECs), and static Symmetry-Breaking constraints are applied to anchor maximum-cardinality jobs. Simultaneously, the online Regression Bounding model is activated to predict pricing objectives, explicitly reducing the exact MILP search trees without sacrificing pure OR optimality guarantees.

3. **Phase 3: Neural Pricing Injection.** The exact MILP oracle is relegated strictly to a fallback mechanism. We deploy the bipartite Graph Neural Network (GNN) to generate localized embeddings of the job-tool dual space, aggressively pruning the hypergraph topology. Attention-based Neural Sequence Oracles are activated to predict and

construct continuous column vectors in milliseconds, bypassing exact H-SUKP feasibility checking entirely during the primary generation loops.

4. **Phase 4: SOTA Synthesis and Global RL Stabilization.** The framework achieves full Neural Column Generation (NCG) synthesis. To combat the severe primal degeneracy and dual oscillations inherent in large-scale set-covering topologies, the Actor-Critic Reinforcement Learning (RL) agent is deployed. The RL agent observes the GNN state embeddings and executes a policy to select a batch of highly diverse, mutually orthogonal configurations. This intelligent, multi-column injection completely stabilizes the dual space, minimizing total CG iterations and defining the new state-of-the-art (SOTA) for exact industrial tool management.

By executing this progressive roadmap, future research can definitively bridge the gap between classical combinatorial rigidity and modern neural approximation, resolving the Job Sequencing and Tool Switching Problem with unparalleled computational efficiency.